Optimization and Thermodynamics

From convergent attractors to algorithmic entropy, and what physics charges an embedded agent for steering the world

Lecture notes, Agent Foundations module
Iliad Intensive

June 2026

Abstract

A useful way to characterize powerful agents is that they reliably steer the world into a narrow region of outcome space, a region that would be extremely unlikely to arise under any random process. These notes develop that picture from first principles, in three stages. First, we make "steering into a narrow region" precise. Following Flint, an optimizing system is a physical system that evolves from a broad basin of attraction toward a small target set, robustly to perturbations; translated into information theory, optimization is local entropy reduction. Second, we ask what physics says about such processes. The reversibility of microscopic dynamics, together with the second law of thermodynamics, forbids global entropy reduction, so an embedded optimizer must balance its books. We derive the second law from reversibility in a minimal Markov-chain setting, classify the three ways an agent can reduce a subsystem's entropy while respecting information conservation, and present the Touchette–Lloyd theorem (entropy reduction beyond a blind baseline must be paid for in mutual information between the agent and the environment), which quantifies the third of those ways. A self-contained toy thermodynamics built out of biased coins (Wentworth's generalized heat engine), in which every step of the bookkeeping is elementary, is developed in an appendix. Third, we confront a conceptual gap. The standard Gibbs–Shannon entropy is defined relative to a subjective probability distribution that the formalism treats as exogenous; this makes "capacity to optimize" observer-dependent and leaves the classical tools inapplicable to the far-from-equilibrium, information-bearing states that agents are made of. Algorithmic thermodynamics, due to Ebtekar and Hutter, replaces the subjective distribution with Kolmogorov complexity, yielding a second law that applies to individual physical states. Under this reframing, an embedded agent's probabilistic knowledge becomes an endogenous, physically encoded quantity: the algorithmic mutual information between the agent's memory and the environment, which is precisely the resource the agent spends when it optimizes. Agents that know more about the world can do more to it, and the exchange rate is set by thermodynamics.

Contents

1 Introduction: why thermodynamics belongs in agent foundations

Agent foundations looks for *robust concepts*, sometimes called "true names", for notions like optimization, goals, world models, and embeddedness: concepts that do not break down when subjected to extreme optimization pressure, and that remain meaningful for agents far more capable than anything we have observed. These notes are about the true name of *optimization* itself. We approach it from the descriptive direction: rather than starting from the subjective viewpoint of an ideal rational agent (beliefs, preferences, expected utility), we start from properties of the physical world and ask what kinds of processes can reliably steer the world into narrow target configurations, and what physics charges them for doing so.

Thermodynamics enters this story for two reasons, and it is worth stating both up front, since the entire document is an unpacking of them.

The first reason is that **optimization is local entropy reduction**. An optimizing process is one that concentrates probability mass: there are many configurations the world could start in, and the process funnels them into a few target configurations. Concentrating probability mass over configurations is the same thing as reducing uncertainty about the configuration, and reduction of uncertainty is, as we will define precisely, reduction of entropy. This identification is not a loose metaphor. It means that the substantial body of physics governing entropy (the second law of thermodynamics, fluctuation theorems, the thermodynamic cost of erasing information) applies directly to optimization processes, and much of that body of physics, in its modern "stochastic thermodynamics" form, holds arbitrarily far from equilibrium. Physics therefore constrains the shape of any optimization process that is actually implemented in the world.

The second reason is that **embedded agents pay for knowledge in physical currency**. Consider the difference between a dualistic agent and an embedded one. A dualistic agent stands outside the world it manipulates, like a player seated outside a video game: it has clean input and output channels, and "what the agent knows" is a free parameter, a probability distribution that we, the modelers, write down. An embedded agent is part of the world it manipulates. Its knowledge cannot be a free parameter, because that knowledge has to be physically encoded somewhere, in a brain or a memory register made of the same atoms as everything else. One of the central conclusions of these notes is that when entropy is defined correctly (algorithmically, as a property of individual physical states, rather than relative to a subjective ensemble), an agent's knowledge about its environment becomes an objective physical quantity, namely the mutual information between the agent's memory and the environment, and this quantity is exactly the budget the agent can spend on optimization.

For readers who already have a physics background, we should be honest that much of the value here lies in translation rather than in new results. Reframing familiar thermodynamic facts in the language of information theory connects them to concepts native to agent foundations, such as world modeling, optimization power, and the physical constraints on embedded agents. For readers without a physics background, the notes are designed to be self-contained: every thermodynamic notion that appears is defined here, in information-theoretic terms, and no prior acquaintance with statistical mechanics is assumed. The only prerequisite is comfort with elementary discrete probability.

Plan of the notes. Section 2 builds the probabilistic and information-theoretic toolkit from scratch: entropy, mutual information, divergence, and the coding interpretations that make these quantities meaningful. Section 3 asks what optimization is, develops Flint's notion of an optimizing system, explains Yudkowsky's proposal for measuring optimization in bits, and translates both into the language of entropy. Section 4

confronts this picture with the reversibility of microscopic physics and derives the second law of thermodynamics, with full proof, in a minimal coarse-grained setting. Section 5 then classifies the three ways an embedded agent can reduce a subsystem's entropy while respecting global information conservation: dumping it into the environment, absorbing it into memory by measurement, or spending pre-existing mutual information with the subsystem. Section 6 presents the Touchette–Lloyd theorem, which makes the third of these quantitative: the entropy reduction an agent can achieve beyond a blind baseline is bounded by the mutual information between its action and the environment, with fully worked examples. Section 7 exposes a conceptual problem at the heart of the standard formalism: entropy, as usually defined, depends on a subjective probability distribution supplied from outside the physics. Section 8 develops the resolution, algorithmic thermodynamics: Kolmogorov complexity as entropy, the algorithmic second law, Landauer's principle, the thermodynamic costs of information processing, an exact analysis of Maxwell's demon, and an information engine that runs on compressible strings. Section 9 assembles the synthesis for embedded agency: knowledge as an endogenous physical resource. Section 10 collects the takeaways. Appendix A develops, in parallel to the main line, a self-contained toy thermodynamics built out of biased coins (Wentworth's generalized heat engine), in which the analogues of heat, work, energy conservation, and the impossibility of perpetual motion can all be verified by hand; the main text points to it wherever a concrete blind-policy example helps.

2 A self-contained toolkit: probability, entropy, information

This section develops everything we need from information theory. Readers who know Shannon entropy and mutual information well can skim it, but should at least note the coding interpretation in Section 2.3, which we lean on repeatedly, and the log-sum inequality (Lemma 2.9), which powers our proof of the second law.

2.1 Random variables and notation

Throughout, capital letters X, Y, Z denote random variables taking values in countable (finite or countably infinite) sets, and lowercase letters x, y, z denote particular values. We write $\mu(x)$ for the probability that $X = x$ when the distribution is called μ, and we freely say "the distribution of X" or "the ensemble" for μ. A joint random variable (X, Y) has a joint distribution, and the conditional probability of y given x is written $P(y \mid x)$. All logarithms in these notes are base 2, so that information is measured in bits; the natural-log versions of every formula differ only by a factor of $\ln 2$.

2.2 Entropy

Definition 2.1 (Shannon entropy)**.** The *Shannon entropy* of a random variable X with distribution μ is

$$H(X) \;:=\; \sum_x \mu(x) \log \frac{1}{\mu(x)},$$

with the convention that terms with $\mu(x) = 0$ contribute zero. We also write $H(\mu)$ for the same quantity when we want to emphasize the distribution rather than the variable.

Entropy measures uncertainty: how much you do not know about the value of X before you observe it. Some examples will calibrate the scale.

Example 2.2 (Calibrating entropy)**.** A fair coin has entropy $\frac{1}{2}\log 2 + \frac{1}{2}\log 2 = 1$ bit. A deterministic variable (one whose value is certain) has entropy 0 bits. A uniform distribution over all binary strings of length n has entropy n bits, and n bits is the maximum possible entropy for a variable with 2^n possible values: uniform distributions are the most uncertain ones. A *biased* coin that lands heads with probability p has entropy $h(p) := p \log \frac{1}{p} + (1-p) \log \frac{1}{1-p}$. Two values of this function will matter later in these notes: $h(0.2) \approx 0.72$ bits and $h(0.1) \approx 0.47$ bits. A coin biased toward one outcome is more predictable than a fair coin, so it carries less than one bit of uncertainty.

2.3 The coding interpretation

The deepest way to understand entropy, and the one we will use constantly, is through data compression. Suppose you must transmit the value of X to a colleague using binary codewords, and you want to minimize the average number of bits sent. The optimal strategy is to give short codewords to likely values and long codewords to unlikely ones. Shannon's source coding theorem makes this precise: there is a code (a prefix-free assignment of binary strings to outcomes) whose codeword for outcome x has length essentially $\log \frac{1}{\mu(x)}$ bits, and no code can achieve a smaller average length than the entropy. Therefore

$$H(X) \;=\; \text{the minimum average number of bits needed to describe a sample of } X.$$

Example 2.3 (A small optimal code)**.** Let X take four values with probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$. Assign the codewords `0`, `10`, `110`, `111`. Each codeword for an outcome of probability $2^{-\ell}$ has length exactly $\ell = \log \frac{1}{\mu(x)}$, and the average length is $\frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + \frac{1}{8}\cdot 3 + \frac{1}{8}\cdot 3 = 1.75$ bits, which equals $H(X)$ exactly.

The quantity appearing inside the expectation deserves its own name, because much of Section 7 turns on the distinction between it and its average.

Definition 2.4 (Shannon codelength, also called stochastic entropy or surprisal)**.** For an individual outcome x under distribution μ, the *Shannon codelength* is

$$\hat{H}(x, \mu) \;:=\; \log \frac{1}{\mu(x)}.$$

It is the length of x's codeword under the code optimized for μ, and the entropy is its mean: $H(\mu) = \Big\langle \hat{H}(X, \mu) \Big\rangle_{X \sim \mu}$.

Notice carefully what is and is not a function of what. The codelength $\hat{H}(x, \mu)$ is a property of the *pair* (outcome, distribution). The same physical outcome x has a short codelength under a distribution that expected it and a long codelength under a distribution that did not. There is, so far, no such thing as "the entropy of an individual outcome x" without reference to some distribution. Keep this in mind; it becomes the crux of Section 7.

2.4 Joint and conditional entropy, and mutual information

Definition 2.5 (Conditional entropy)**.** For jointly distributed X, Y, the *conditional entropy* of X given Y is

$$H(X \mid Y) \;:=\; H(X, Y) - H(Y),$$

where $H(X, Y)$ is the entropy of the pair. Equivalently, $H(X \mid Y)$ is the average, over values y, of the entropy of the conditional distribution of X given $Y = y$. It measures the uncertainty that remains about X once Y is known.

The rearranged identity $H(X, Y) = H(Y) + H(X \mid Y)$ is called the *chain rule*, and it has a transparent coding reading: to describe the pair, first describe Y (costing $H(Y)$ bits on average), then describe X using a code adapted to the known value of Y (costing $H(X \mid Y)$ bits on average).

Definition 2.6 (Mutual information)**.** The *mutual information* between X and Y is

$$I(X; Y) \;:=\; H(X) + H(Y) - H(X, Y) \;=\; H(X) - H(X \mid Y) \;=\; H(Y) - H(Y \mid X).$$

Mutual information is the number of bits that knowing Y saves you, on average, when describing X; by the symmetry of the definition, it is also the number of bits knowing X saves you about Y. It is zero exactly when X and Y are independent, and it is never negative (we prove this below). It is the standard measure of "how much one part of the world knows about another", and that informal phrase will recur many times in these notes.

Example 2.7 (Mutual information, calibrated)**.** Let $X = (B_1, B_2)$ consist of two independent fair bits, and let $Y = B_1$ be the first of them. Then $H(X) = 2$, and once you know Y only the second bit remains uncertain, so $H(X \mid Y) = 1$ and $I(X; Y) = 1$ bit: Y contains exactly one bit of information about X. If instead Y were an independent coin flip, $I(X; Y)$ would be 0; if Y were a full copy of X, $I(X; Y)$ would be 2 bits, the whole entropy of X.

2.5 Divergence and two workhorse inequalities

Definition 2.8 (Kullback–Leibler divergence)**.** For two distributions μ, ν on the same countable set,

$$D(\mu \,\|\, \nu) \;:=\; \sum_x \mu(x) \log \frac{\mu(x)}{\nu(x)}.$$

In coding terms, $D(\mu\|\nu)$ is the price of being wrong about the source: if the truth is μ but you compress using the code optimized for ν, you spend on average $H(\mu) + D(\mu\|\nu)$ bits instead of $H(\mu)$. The definition makes sense, and we will use it, even when the reference ν is an unnormalized nonnegative measure rather than a probability distribution.

Both of the inequalities we need follow from a single elementary fact about the convex function $t \mapsto t \log t$.

Lemma 2.9 (Log-sum inequality)**.** *Let $a_1, a_2, \ldots$ and $b_1, b_2, \ldots$ be nonnegative reals with finite sums $a := \sum_i a_i$ and $b := \sum_i b_i > 0$. Then*

$$\sum_i a_i \log \frac{a_i}{b_i} \;\geq\; a \log \frac{a}{b},$$

with the conventions $0 \log \frac{0}{b} = 0$ and $a \log \frac{a}{0} = +\infty$ for $a > 0$.

Proof. The function $\varphi(t) = t \log t$ is convex on $[0, \infty)$. Apply Jensen's inequality (for convex φ, the average of φ is at least φ of the average) with weights $w_i = b_i / b$ and points $t_i = a_i / b_i$:

$$\sum_i \frac{b_i}{b} \varphi\Big(\frac{a_i}{b_i}\Big) \;\geq\; \varphi\Big(\sum_i \frac{b_i}{b} \cdot \frac{a_i}{b_i}\Big) \;=\; \varphi\Big(\frac{a}{b}\Big).$$

Multiplying both sides by b and simplifying $b_i \cdot \frac{a_i}{b_i} \log \frac{a_i}{b_i} = a_i \log \frac{a_i}{b_i}$ gives the claim. □

Taking all the a_i and b_i to be the probabilities of two distributions ($a = b = 1$) yields *Gibbs' inequality*: $D(\mu\|\nu) \geq 0$, with equality only when $\mu = \nu$. Nonnegativity of mutual information follows because $I(X;Y) = D\big(\text{joint distribution} \,\|\, \text{product of marginals}\big) \geq 0$. The second consequence, which we prove when we need it in Section 4, is the *data processing inequality* for divergence: pushing two distributions through the same noisy channel can only bring them closer together.

With the toolkit in hand, we can turn to the actual subject.

3 What is optimization?

3.1 Two everyday notions, and the question that links them

In computer science, an optimization algorithm is a program that outputs the solution, or an approximation of the solution, to an optimization problem: an objective function to be minimized over a feasible region. A program that returns $x \approx \sqrt{2}$, the minimizer of $y = (x^2 - 2)^2$ over the real line, is performing optimization in this sense. In operations research and engineering, optimization means something physically heftier: improving a process or artifact so that it better fits a purpose, as when a factory is reorganized to produce more nails per unit cost. There is clearly a connection between optimizing a variable inside a computer and optimizing a factory, but what exactly is it? What property does a physical process have to have for us to rightly call it an optimization process?

3.2 Optimizing systems

Flint's answer, in *The ground of optimization*, is to refuse the usual decomposition into an "optimizer" acting on an "optimized". Instead, we consider the whole physical system jointly (the engine of optimization together with its object, the program together with the computer, the builders together with the building site) and ask whether the whole exhibits a characteristic tendency.

Definition 3.1 (Optimizing system, Flint 2020)**.** An *optimizing system* is a physical system with the following property: there is a small set of *target configurations* and a much larger set of configurations, the *basin of attraction*, such that whenever the system starts anywhere in the basin of attraction, it tends to evolve toward the target configurations, and it continues to exhibit this tendency *despite perturbations* applied while the process is underway.

The definition has three load-bearing parts (a broad basin, a narrow target, and robustness to perturbation), and the best way to absorb them is through examples.

Example 3.2 (Computing $\sqrt{2}$ by gradient descent)**.** A computer runs a few lines of code implementing gradient descent on the objective $y = (x^2 - 2)^2$: starting from an initial guess stored in a variable `current_estimate`, the program repeatedly computes the slope of the objective at the current estimate and takes a small step in the downhill direction, stopping when the slope is nearly zero. The program converges to $\sqrt{2}$.

Here is the remarkable property. Suppose that while the loop is running, you attach a debugger and overwrite `current_estimate` with an arbitrary number. The algorithm does not crash, and it does not output a wrong answer. The next iteration computes the slope at the new point, and every subsequent step pushes the estimate back toward $\sqrt{2}$; given time to converge, the output is bit-for-bit identical to what it would have been without the interference. Most computer code is nothing like this. If you drop into a webserver mid-request and overwrite a loop variable, you get a crash or a corrupted response. The gradient descent system (computer plus program) has a broad basin of attraction (every representable floating-point value of the estimate), a tiny target set (the configuration where the estimate equals $\sqrt{2}$), and a tendency to evolve from the former to the latter that survives in-flight perturbation. It is an optimizing system, though only along the dimensions of perturbation corresponding to the estimate variable; scramble the program's machine code and the tendency is destroyed.

Example 3.3 (A ball in a valley, and billiard balls)**.** A ball rolling in a valley, with friction, ends up at the bottom of the valley no matter where in the valley it starts, and no matter how it is nudged along the way (so long as it is not knocked clear out of the valley). The system consisting of the valley and the ball is an optimizing system: broad basin (anywhere in the valley), narrow target (the set of local minima), robust tendency. It is a *weak* optimizing system, robust along few dimensions (the ball's position and velocity) and against modest perturbations, but it is a genuine one.

By contrast, consider billiard balls bouncing energetically around a table. Move one ball a few inches while they are in motion and the final resting configuration is utterly different from what it would have been. There is no small target set toward which the system reliably evolves; the system is not an optimizing system. The same is true of a satellite in orbit: alter its trajectory slightly and it has no tendency whatsoever to return to the old orbit. Most physical systems are like the billiard balls and the satellite. Systems with robust convergent tendencies are special, and that specialness is what Definition 3.1 isolates.

Example 3.4 (Building a house)**.** Seal inside a closed chamber a team of humans with appropriate instructions and incentives, construction materials, a blueprint, food, lighting, and sleeping quarters. This physical system, evolving under nothing but the ordinary laws of physics, tends toward configurations in which there is a house matching the blueprint. If we perturb it (sneaking in at night to remove a wall or scatter the lumber), the system recovers: the next day the team finds the moved materials and rebuilds the missing wall. The basin of attraction (all arrangements of viable humans and intact materials) is enormous compared to the target set (configurations containing the finished house). The system is not infinitely robust; scramble the atoms thoroughly enough and there are no builders left and the tendency is gone. But within its basin, it is a paradigm optimizing system.

A perturbation that pushes an optimizing system beyond the outer rim of its basin destroys the convergent tendency, and Flint suggests we say the optimizing system *dies*. A tree continues growing toward one of its mature configurations through droughts and broken branches, but not through a forest fire. (In this vocabulary, an existential catastrophe is the event in which the optimizing system of life on Earth is perturbed beyond its own outer rim.)

3.3 Three axes for comparing optimizing systems

Flint proposes to quantify and compare optimizing systems along three axes, each of which will reappear, in thermodynamic costume, by the end of these notes.

Robustness. Along how many dimensions can the system be perturbed without destroying its convergent tendency, and how large a perturbation can it absorb along each? The ball in the valley is robust only to repositioning of the ball. The house-building system is robust to a vastly larger family of disruptions. A self-driving car navigating to a destination is robust to being physically relocated within the city, but not to the flipping of critical bits in its memory.

Duality. To what extent can we factor the system into "that which optimizes" and "that which is optimized"? A system consisting of a robot arranging a vase is highly dualistic: there is a clearly identifiable engine of optimization (the robot) and object of optimization (the vase), and characteristically, the system is robust to perturbations of the object (move the vase and the robot fetches it) but brittle to perturbations of the engine (cut one wire inside the robot and the whole tendency vanishes). A tree, on the other hand, is a *non-dualistic* optimizing system: it robustly grows toward a mature configuration, but there is no part of the tree that is "doing the optimizing" while the rest "gets optimized". Optimization, in Flint's sense, does not require an agent. This deliberate widening of scope matters for what comes later: the thermodynamic constraints we derive apply to optimizing systems as such, agent-shaped or not.

Retargetability. Is there a way to redirect the system toward a *different* target configuration set using only a microscopic perturbation? Flipping a small number of bits in the robot's memory makes the whole robot-plus-vase system converge to a different vase position; the system is retargetable. No microscopic nudge will make the ball roll to a different valley; the ball system is not retargetable. Retargetability via small, localized perturbations is a signature of something agent-like: it indicates that the target is stored in a compact representation somewhere inside the system, which is what makes the difference between a system that merely *has* a convergent tendency and one that *represents* a goal.

3.4 Measuring optimization in bits

Definition 3.1 is qualitative. To connect it to physics we need numbers, and the natural numbers to use were proposed by Yudkowsky in *Measuring optimization power*, several years before Flint's essay.

Yudkowsky's proposal runs as follows. Suppose we look at a region of the world that has been acted on by an optimization process, and we observe the final configuration it reached. We can rank all the configurations the region could have been in according to the optimizer's preference ordering, and then ask: *if the region had instead been arranged at random (with no optimization), what fraction of outcomes would have been at least as high in the preference ordering as the outcome actually achieved?* If that fraction is 2^{-k}, we say the process exerted k *bits of optimization power*. Each additional bit corresponds to halving the fraction of equally-good-or-better outcomes, that is, to hitting a target twice as narrow. A configuration of a room that only one arrangement in 2^{30} would match or beat represents thirty bits of optimization.

This measure captures the "narrow region of outcome space" intuition from the opening sentence of these notes in a precisely quantified way: powerful optimization means reliably producing outcomes that random processes would essentially never produce. Flint's definition differs from Yudkowsky's in two deliberate respects, both of which we inherit. First, Yudkowsky's measure looks at a patch of the world and presumes a separate mind whose preference ordering defines the ranking, whereas Flint considers whole closed systems with no assumed engine/object split. Second, where Yudkowsky's measure looks at the improbability of the final configuration, Flint's definition emphasizes *robustness*, the persistence of the convergent tendency under perturbation, as the feature that distinguishes genuine optimization from lucky accident. The two views are complementary: robustness explains *why* an optimizing system reliably produces improbable outcomes, and improbability measures *how much* optimization the system achieves.

3.5 Optimization as entropy reduction

We now have all the pieces needed to translate optimization into the language of Section 2, which is the bridge to physics.

Model the configuration of the relevant subsystem of the world as a random variable: X for the initial configuration and Y for the final one. The randomness in X represents the breadth of the basin of attraction: there are many configurations the system might begin in, and an observer who knows only that the system starts somewhere in the basin has high uncertainty $H(X)$. An optimizing system funnels this broad set of possibilities into a narrow target set, so the same observer's uncertainty about the final configuration, $H(Y)$, is small. The natural quantitative summary of how much funneling occurred is the *entropy reduction*

$$\Delta H \;:=\; H(X) - H(Y).$$

Yudkowsky's bits of optimization power and the entropy reduction ΔH are two faces of the same idea: both count the bits by which the process narrows the world's possibilities. Robust convergence from a broad basin means $H(X)$ can be large while the convergent tendency still operates; a narrow target set means $H(Y)$ is small; and the phrase "an outcome that would be extremely unlikely to arise under any random process" says exactly that the target set has low probability under unoptimized dynamics, so that reliably reaching it constitutes a large ΔH.

Remark 3.5 (What the translation keeps and what it drops). The summary ΔH deliberately forgets the *direction* of optimization: it records how narrow the achieved region is, not which region it is or whether anyone wanted it. This is less of a loss than it may appear. Wentworth has observed that any expected utility maximization problem can be decomposed into two parts: an entropy-minimization part, and a part that makes the world's distribution resemble one particular target distribution (in his slogan, "utility maximization is description length minimization"). These notes are about the first part, which is where the laws of physics impose their constraints; the second part (which target, and why) is the business of the theory of goals and preferences, not of thermodynamics.

So: optimization is local entropy reduction. The word "local" is about to do an enormous amount of work, because, as the next section shows, *global* entropy reduction is physically impossible.

4 Reversibility and the second law

4.1 Microscopic physics is reversible

At the most fundamental level we know how to describe it, the world does not forget. In classical mechanics, the complete microscopic state of a system (the *microstate*) is the list of positions and momenta of all its particles, a single point in the system's *phase space*, and the dynamics carry each microstate along a uniquely determined trajectory: the laws of motion determine the future from the present, and equally determine the past from the present. Two distinct present microstates can never evolve into the same future microstate, because if they did, running the laws backward from that future state would be ambiguous, and the laws of mechanics do not permit ambiguity in either direction. The time evolution of an isolated system over any fixed interval is therefore a *bijection* on microstates: a perfect, invertible pairing of initial states with final states. Classical mechanics adds a further refinement known as Liouville's theorem: time evolution preserves not just the distinctness of states but the *volume* of regions of phase space, so a blob of possible microstates may be stretched and folded into complicated shapes, but its total volume never changes. Quantum mechanics tells the same story in different mathematics: the evolution of an isolated system is unitary, which again means invertible and measure-preserving. In either framework, microscopic information is never created and never destroyed.

4.2 Why the funneling picture cannot hold globally

Now hold the convergent-attractor picture of optimization up against this reversibility, and observe that, taken literally at the microscopic level, they directly conflict. Funneling means many initial states end up in the same few target states: a many-to-one map. Reversibility says the microscopic dynamics are one-to-one. A perfect funnel is microscopically impossible.

There is also a second, closely related obstruction, which is the second law of thermodynamics itself: the entropy of an isolated system tends not to decrease. In fact, within the framework we are about to set up, the second law is not an additional postulate but a *consequence* of reversibility, and seeing why illuminates both. The resolution of the apparent paradox (optimization manifestly happens, yet physics forbids funneling) is the central bookkeeping principle of these notes:

> *No physical process can reduce the entropy of an isolated system. Optimization is entropy reduction in a coarse-grained description of a subsystem, and the information squeezed out of the optimized subsystem must be accounted for somewhere else in the larger system.*

The remainder of this section makes the first sentence precise and proves it. Sections 5 and 6 take up the second sentence: where the squeezed-out information can go, and how much squeezing an agent can do.

4.3 Coarse-graining: where probability enters a deterministic world

If microscopic dynamics are deterministic and information-preserving, where do probability and entropy increase come from? The answer is that we never track the microstate. The microstate of a gram of matter involves on the order of 10^{23} coordinates, specified to unbounded precision; no observer, and no embedded agent, has access to it. What we track instead is a *coarse-grained* description: we partition the phase space into discrete cells, lumping together all microstates that agree on what we can resolve (say, every particle's position and momentum to finitely many digits, or the values of a few macroscopic variables), and we describe the system by naming the cell it currently occupies.

The dynamics of cells, unlike the dynamics of microstates, are genuinely stochastic. Knowing the current cell does not determine the next cell, because different microstates within the cell flow to different places; the best available description is the transition probability $P(y, x)$, defined as the fraction of cell x (as measured by phase-space volume) whose evolution lies in cell y after one time step. The randomness here is not metaphysical; it is the shadow of the unresolved microscopic detail. Chaotic dynamics continually pull information up from below the resolution scale, so the coarse-grained trajectory looks like a sequence of random transitions even though the underlying flow is deterministic.

The key structural assumption, which underlies all of stochastic thermodynamics, is that the coarse-grained trajectory is a *Markov process*: the probability distribution of the next cell depends only on the current cell, and not additionally on the history of earlier cells. Intuitively, this says the coarse-graining is chosen well enough that the cell captures everything about the past that matters for predicting the coarse future; the discarded microscopic details behave like fresh noise at each step rather than like a hidden memory. Whether a given coarse-graining is (approximately) Markovian is a deep question that remains an active research area, but two reassurances are available. First, there are exactly solvable model systems, the *multibaker maps*, which are deterministic, time-reversible, chaotic dynamical systems whose coarse-grainings provably emulate arbitrary Markov chains, with randomness residing only in the initial condition. These models rigorously demonstrate that macroscopic stochasticity and irreversibility are compatible with microscopic determinism and reversibility. Second, the Markov assumption quietly underwrites the everyday statistical regularities we all rely on, and its *failure in the time-reversed direction* is the arrow of time. A falling glass vase shatters at a predictable moment, and the distribution of its shards follows well-defined local statistics that do not depend on anything happening at the neighbor's house. Run the film backward and the situation is completely different: to "retrodict" when a shattered vase was dropped, local statistics are useless, and your best evidence might be a conversation about the accident taking place next door. Forward evolution obeys memoryless local statistical laws; backward evolution does not. That asymmetry is what the Markov property encodes.

One more consequence of reversibility is needed before we can state the second law. Liouville's theorem says phase-space volume is conserved, and the volume of each cell therefore behaves as a *stationary measure* of the Markov chain: if we weight each cell by its volume, one step of the dynamics carries that weighting to itself. In these notes we make the simplifying assumption (called a *mesoscopic* coarse-graining) that all cells have equal volume, as happens when cells are specified by truncating every coordinate to fixed precision. Stationarity of the uniform weighting then says something concrete about the transition probabilities. Writing the stationarity condition $\sum_x P(y, x)\, \pi(x) = \pi(y)$ with π constant and cancelling π gives $\sum_x P(y, x) = 1$ for every y: the transition probabilities *into* each cell sum to one, just as the transition probabilities *out*

of each cell do. A transition matrix with both properties is called *doubly stochastic*. Double stochasticity is the fingerprint that microscopic reversibility leaves on the coarse-grained dynamics: a doubly stochastic step can shuffle and spread probability, but it cannot concentrate it, because concentration would require squeezing several cells' worth of volume into fewer cells, which Liouville forbids.[1]

4.4 The second law of thermodynamics

Theorem 4.1 (Second law for doubly stochastic chains)**.** *Let P be a doubly stochastic transition matrix on a countable state space, let the random variable X (the coarse-grained state now) have distribution μ, and let Y (the state one step later) have distribution $P\mu$, where $(P\mu)(y) := \sum_x P(y,x)\,\mu(x)$. Then*

$$H(Y) \;\geq\; H(X).$$

Proof. We prove the statement in two steps, each of independent interest.

Step 1: one step of any Markov chain brings distributions closer together. Let μ and ν be any two distributions (or nonnegative measures) on the state space. We claim the data processing inequality

$$D(P\mu \,\|\, P\nu) \;\leq\; D(\mu \,\|\, \nu).$$

To see this, fix an output state y and apply the log-sum inequality (Lemma 2.9) to the numbers $a_x = P(y,x)\mu(x)$ and $b_x = P(y,x)\nu(x)$, whose sums over x are $P\mu(y)$ and $P\nu(y)$ respectively:

$$\sum_x P(y,x)\mu(x)\log\frac{P(y,x)\mu(x)}{P(y,x)\nu(x)} \;\geq\; P\mu(y)\log\frac{P\mu(y)}{P\nu(y)}.$$

Now sum this inequality over all y. On the left-hand side, the factor $\log\frac{\mu(x)}{\nu(x)}$ does not depend on y, and $\sum_y P(y,x) = 1$ because P is stochastic, so the left-hand side totals $\sum_x \mu(x)\log\frac{\mu(x)}{\nu(x)} = D(\mu\|\nu)$. The right-hand side totals $D(P\mu\|P\nu)$. This proves the claim. The intuition is exactly the coding one: passing two information sources through the same noisy channel can only make them harder to tell apart.

Step 2: choose the reference measure to be uniform. Let $\sharp$ denote the counting measure, $\sharp(x) = 1$ for every x (an unnormalized uniform measure; Lemma 2.9 never required normalization). Double stochasticity of P says precisely that $P\sharp = \sharp$, since $(P\sharp)(y) = \sum_x P(y,x) = 1$. Moreover, divergence from the counting measure is just negative entropy:

$$D(\mu \,\|\, \sharp) \;=\; \sum_x \mu(x)\log\frac{\mu(x)}{1} \;=\; -H(\mu).$$

Applying Step 1 with $\nu = \sharp$:

$$-H(P\mu) \;=\; D(P\mu \,\|\, P\sharp) \;\leq\; D(\mu \,\|\, \sharp) \;=\; -H(\mu),$$

which rearranges to $H(P\mu) \geq H(\mu)$. □

It is worth pausing on what this proof actually used, because the ingredient list is short and each item is physical. We used the Markov property (to model one step as a transition matrix), and we used double stochasticity, which we obtained from Liouville's theorem, that is, from the *reversibility* of the microscopic dynamics. Nothing else. Macroscopic irreversibility (entropy increase) is therefore not in tension with microscopic reversibility; it is a *consequence* of microscopic reversibility, once we account for the fact that observers track coarse-grained cells rather than microstates. This is the precise content of the slogan "the second law follows from the reversibility of physics".

[1] Everything in these notes generalizes to coarse-grainings with unequal cell volumes: one measures entropy *relative to* the stationary volume measure π, replacing $H(\mu)$ by $H_\pi(\mu) := \sum_x \mu(x)\log\frac{\pi(x)}{\mu(x)}$ and the codelength by $\hat{H}_\pi(x,\mu) := \log\frac{\pi(x)}{\mu(x)}$. Unequal volumes become essential when modeling heat reservoirs, and we will say so explicitly at that point (Section 8.4); until then, equal cells keep the notation light at no conceptual cost.

4.5 What the second law does and does not forbid

Three clarifications will prevent misreadings that matter later.

First, Theorem 4.1 applies to an *isolated* system evolving on its own: the entropy of the whole cannot fall. It says nothing against the entropy of a *subsystem* falling, and the entropy of subsystems falls all the time; that is what refrigerators, crystallization, house-builders, and every other optimizing system accomplish. What the theorem imposes is a bookkeeping constraint on the whole: when a subsystem's entropy drops, the books must balance, with at least as much entropy appearing elsewhere in the total system. The complete classification of where "elsewhere" can be is the subject of Section 5.

Second, the theorem as stated concerns the entropy of the *ensemble*, the probability distribution μ, and it holds on average and in distribution; individual trajectories can pass through improbable, low-codelength states. A sharper, trajectory-level version of the second law, with explicit and very small bounds on the allowed fluctuations, becomes available in the algorithmic framework of Section 8.

Third, and most importantly for what follows: the theorem quietly assumed a fixed dynamics P uninfluenced by any observer, and an entropy defined relative to a given distribution μ. Agents complicate both assumptions. An agent that *observes* the system and conditions its behavior on what it sees is not a fixed P; how much that buys, and what it costs, is the subject of Section 6. And the apparently innocent reliance on a given μ conceals a deep conceptual problem, the subject of Section 7. But first we take up the bookkeeping question directly: when an agent lowers a subsystem's entropy, where can the displaced entropy go? The next section gives the complete answer. (A self-contained toy model in which every step can be checked by hand is developed separately in Appendix A.)

5 Three types of optimization under information conservation

5.1 The bookkeeping problem

We now assemble the global accounting. Decompose the universe into three parts: the agent A, the subsystem S that the agent is trying to optimize, and the rest of the environment E. Optimization of S means reducing $H(S)$: funneling the subsystem from many possible configurations toward a narrow target set. The second law (Theorem 4.1, applied to the isolated whole) says the joint entropy of (A, S, E) cannot decrease, and the reversibility that underlies it says information about S's initial condition cannot simply vanish. So the bookkeeping question is: in what ways can an agent reduce $H(S)$ while the global books remain balanced? Daniel C and Ebtekar's analysis identifies exactly three channels, and the classification is exhaustive: entropy squeezed out of S must end up in E, end up in A, or have been pre-paid.

5.2 Type 1: dump waste heat into the environment

The first channel is the everyday one: transfer the subsystem's entropy into the environment. The agent arranges an interaction between S and E under which $H(S)$ falls while $H(E)$ rises by at least as much. A refrigerator narrows the distribution of its interior's thermal state while heating the kitchen behind it. A house-building team imposes order on lumber and nails while dissipating heat, exhaust, and scattered debris into the surroundings. In the coin world of Appendix A, this is a conditional-swap circuit that moves uncertainty from designated coins into other coins. The signature of Type 1 is that the environment absorbs the entropy: the agent is a broker arranging the transfer, and the agent's own state need not change appreciably.

5.3 Type 2: absorb the entropy into the agent's memory (measurement)

The second channel is subtler: the agent reduces $H(S)$ by *measuring* S and storing the outcomes, so that the subsystem's entropy is transferred into the agent's own memory. This is the operating principle of one of the most famous thought experiments in thermodynamics, and it deserves a careful telling.

Maxwell's demon. A room full of gas is divided in two by a partition, and the partition has a small door that can be opened and closed at will. A tiny intelligence (the "demon") watches the molecules. Whenever

a molecule approaches the door from the left, the demon opens it; whenever one approaches from the right, the demon closes it. Molecule by molecule, the gas accumulates on the right side of the room. The entropy of the gas has plainly decreased (we are far less uncertain about the molecules' whereabouts than before), no work appears to have been done, and no heat has been dumped anywhere. Has the second law been violated?

It has not, and the reversibility argument of Section 4.1 shows exactly where the missing entropy went. Consider the end of the process and pick any particle now on the right side. Two distinct histories are consistent with its presence there: either it began on the right and stayed, or it began on the left and the demon admitted it. These two histories produce *identical* final gas configurations, so the gas alone cannot distinguish them. But the microscopic dynamics of the whole system are reversible, and reversibility demands that the past be reconstructible from the present. The only way the total system can remain reversible is if something else differs between the two histories, and that something is the demon's memory: the record of its observations and door operations. For every bit of entropy the demon removes from the gas, at least one bit of entropy accumulates in the demon's memory. The gas's order is purchased with the disorder of the demon's records; globally, nothing has decreased. (We will redo this argument with full rigor, for individual states rather than ensembles, in Section 8.6.)

The signature of Type 2 is that the agent absorbs the entropy: measurement moves uncertainty from the world into the measurer. The demon's memory is finite, so this channel is exhaustible. When the memory fills, the demon must either stop or clear memory, and clearing memory, as we will see with Landauer's principle in Section 8.4, converts the stored entropy into Type-1 waste heat after all. Type 2 is a buffer, not a bottomless sink.

5.4 Type 3: spend mutual information you already have

The first two channels balance the books by increasing entropy somewhere (in E, or in A). The third channel is the surprising one: it reduces $H(S)$ *without increasing entropy anywhere*, by consuming a correlation that already exists between the agent and the subsystem.

Suppose that at time t, the agent's state and the subsystem's state share mutual information $I(S_t; A_t) > 0$: the agent already "knows something" about the subsystem, in the purely statistical sense of Definition 2.6. For clean accounting, assume the environment is independent of both, so that the joint entropy decomposes (by the definition of mutual information and the chain rule) as

$$H(S_t, A_t, E_t) \;=\; H(A_t) \;+\; H(S_t) \;-\; I(S_t; A_t) \;+\; H(E_t).$$

Now the agent performs an operation with the following effects: it *erases the copy of the shared information that resides inside* S, using its own copy as the key, while leaving its own marginal state and the environment untouched. The operation transforms the subsystem so that

$$H(S_{t+1}) \;=\; H(S_t) - I(S_t; A_t), \qquad I(S_{t+1}; A_{t+1}) \;=\; 0, \qquad H(A_{t+1}) = H(A_t), \qquad H(E_{t+1}) = H(E_t).$$

Adding up the joint entropy after the operation:

$$H(A_{t+1}) + H(S_{t+1}) + H(E_{t+1}) \;=\; H(A_t) + H(S_t) - I(S_t; A_t) + H(E_t),$$

which is *exactly* the joint entropy from before. The subsystem's entropy has genuinely fallen by $I(S_t; A_t)$ bits, total entropy is unchanged, no heat was produced, and no memory filled up. What was consumed is the correlation itself: afterward, $I(S_{t+1}; A_{t+1}) = 0$, and the trick cannot be repeated without first re-acquiring mutual information.

Example 5.1 (Type 3 in one bit: the controlled-NOT)**.** The smallest possible instance makes the bookkeeping transparent. Let the subsystem hold a single uniformly random bit s, and let the agent hold a perfect copy, $a = s$. Then $H(S) = 1$, $H(A) = 1$, $I(S; A) = 1$, and the joint entropy is $H(S, A) = 1$ bit (two equally likely joint states, 00 and 11). The agent now applies a *controlled-NOT*: flip s if $a = 1$, do nothing if $a = 0$. In both joint states the result sets s to 0: the subsystem becomes deterministic, $H(S_{t+1}) = 0$, a full bit of entropy reduction. The operation is reversible (apply it again to undo it), touches no environment, and produces no heat. And the joint entropy is still exactly 1 bit, now carried entirely by the agent's own marginal (a remains a uniform bit, uncorrelated with the now-deterministic s). The correlation was spent: a second application of the trick is useless, since $I(S_{t+1}; A_{t+1}) = 0$.

5.5 The demon reframed, and the recap

The Type 3 channel recasts Maxwell's demon in an illuminating way. The demon's measure-and-control behavior can be decomposed into two distinct steps: a *copy* step, in which the demon interacts with the gas to acquire mutual information with it (this is Type 2: the demon's memory absorbs entropy, or more precisely, becomes correlated with the gas), followed by a *control* step, in which the demon uses that mutual information to steer the gas into a narrower distribution (this is Type 3: the correlation is spent, and the control step itself produces no further entropy anywhere). Measurement is the purchase of correlation; steering is its expenditure. How much entropy reduction can the control step buy per bit of correlation acquired in the copy step? That is exactly the question answered by the *Touchette–Lloyd theorem* of the next section: it bounds the steering achievable in a Type-3 step by the mutual information available to spend, bit for bit. (Section 8.6 will later derive an exact algorithmic version of the same bound.)

To summarize, an embedded agent can optimize a subsystem in exactly three ways:

1. **Type 1:** dump the subsystem's entropy into the environment as waste heat;
2. **Type 2:** absorb the subsystem's entropy into its own memory through measurement;
3. **Type 3:** spend mutual information it already shares with the subsystem, erasing the subsystem's copy of the correlation.

The three channels compose: a realistic optimizer cycles among them, measuring (Type 2), steering (Type 3), and periodically flushing its memory to the environment (Type 1) to make room for the next round.

Type 3 raises a quantitative question that the next section takes up. If steering is paid for by spending mutual information shared with the subsystem, exactly how many bits of entropy reduction does each bit of mutual information buy? The Touchette–Lloyd theorem gives the bound.

6 Steering costs information: the Touchette–Lloyd theorem

6.1 From optimization to modeling: the question

In Section 5 we saw that an agent can steer a subsystem by spending mutual information it already shares with it (the Type-3 channel). This section quantifies that channel: how much entropy reduction does each bit of mutual information actually buy? The answer turns out to connect optimization to *modeling*.

One of the recurring hopes of agent foundations is to find *selection theorems*: results stating that any system selected to perform sufficiently well at some task must, as a matter of mathematical necessity, contain certain agent-like structures, such as a world model, a goal representation, or a planning process. The *agent structure problem* asks this in the converse direction to the usual one: instead of "agents bring about outcomes", it asks "if we observe that something reliably brings about a particular outcome, can we conclude that it must be modeling its environment?" Wentworth posed a sharp version of the question: how many bits of optimization can one bit of observation unlock?

A theorem of Touchette and Lloyd, originally published in the control theory literature and brought to the attention of the agent foundations community by Harwood and Altair, is one of the few existing results that directly addresses this problem.[2] Informally, the theorem says: *the entropy reduction a policy achieves, beyond what the environment's dynamics would do on their own, is bounded by the mutual information between the policy's action and the environment's state.* Every bit of optimization beyond the blind baseline requires a bit of modeling. This section builds up the exact statement, with fully worked examples.

[2]The result appears as Theorem 10 of H. Touchette and S. Lloyd, *Information-theoretic approach to the study of control systems*, Physica A 331:140–172 (2004); it is also described in their 2000 paper *Information-theoretic limits of control*, and an earlier, more physics-flavored proof appears in Lloyd's 1989 work on the use of mutual information to decrease entropy, in the context of Maxwell's demon. Related results connecting regulation to modeling include the good regulator theorem of Conant and Ashby and the internal model principle of control theory, but those concern conditions for *optimal* regulation, whereas the Touchette–Lloyd theorem is an inequality constraining *all* policies, optimal or not, which is what makes it feel like a genuine selection theorem.

6.2 The setup: environments, actions, policies

The model is a single time step of an environment under influence. Three random variables appear:

- X, the *initial state* of the environment;
- A, the *action* taken (by an agent, a controller, a machine; the formalism does not care);
- Y, the *final state* of the environment.

Two conditional distributions specify the situation. The *policy* $P(A \mid X)$ describes how the action is chosen as a function of the environment's initial state, and the *dynamics* $P(Y \mid X, A)$ describe how the final state is produced from the initial state together with the action. The dynamics are held fixed (they are the physics of the environment); the policy is the object of study. As is conventional, X and Y range over the same set of environment states, and the dynamics may be deterministic (all transition probabilities 0 or 1) or noisy.

Following Section 3.5, a policy is scored by the entropy reduction it achieves:

$$\Delta H \;:=\; H(X) - H(Y),$$

the number of bits by which the final state is more predictable than the initial state was. We are measuring optimization exactly as Section 3 taught us to: by how much the process funnels a broad distribution into a narrow one. (And as noted in Remark 3.5, entropy reduction is the physics-facing half of expected utility maximization, the other half being which narrow region the agent prefers.)

6.3 Blind and sighted policies

Definition 6.1 (Blind and sighted policies)**.** A policy is *blind* if the action is statistically independent of the initial state, that is, if $P(A \mid X) = P(A)$, or equivalently $I(X; A) = 0$. A policy is *sighted* if $I(X; A) > 0$.

Blind policies include every deterministic rule of the form "always take action a_1", and also every randomized rule whose randomness is independent of the environment, such as "flip a private coin; on heads do a_1, on tails do a_2". What blindness excludes is precisely any flow of information from the environment's state into the choice of action. Sightedness is a matter of degree, and the degree is measured by $I(X; A)$: a policy that conditions on one observed bit of the state has $I(X; A) \leq 1$; a policy that observes everything can have $I(X; A)$ as large as $H(X)$. The whole transformation toolkit of the coin world (Appendix A) consisted of blind policies: the no-peeking rule was exactly the requirement $I(X; A) = 0$, with the "action" being the choice of transformation.

One might naively conjecture that any entropy reduction at all requires sight, but this is false, and seeing why sharpens the eventual theorem. The dynamics alone can reduce entropy: a contracting dynamics, like the ball-in-valley system of Example 3.3, funnels states all by itself, whatever the action; and the coin engine of Appendix A.5 reduced the entropy of designated coins while completely blind. The right question is therefore not whether a policy reduces entropy, but whether it reduces entropy *more than blindness allows*. Accordingly, define the *blind baseline*

$$\Delta H^{\max}_{\text{blind}} \;:=\; \max_{P(X)\in\mathcal{X},\; P(A)\in\mathcal{A}} \Delta H,$$

the largest entropy reduction achievable by *any* blind policy from *any* initial distribution, where $\mathcal{X}$ is the set of all distributions over initial states and $\mathcal{A}$ is the set of all action distributions independent of X. This baseline is a property of the dynamics alone: it is everything the environment can be made to do "on its own".

6.4 A worked example: the guessing game, played blind

The following game, from Harwood and Altair's exposition, makes every quantity in the theorem concrete and computable.

A computer secretly picks a 5-bit binary string X, uniformly at random, so $H(X) = 5$ bits. You then submit a 5-bit string A of your own. The computer feeds both strings into the fixed, publicly known function

$$f(x, a) \;=\; \begin{cases} \texttt{00000} & \text{if } a = x \quad \text{(you matched the secret string)}, \\ x & \text{otherwise}, \end{cases}$$

and outputs $Y = f(X, A)$. Your goal is to make the output distribution as predictable as possible, that is, to minimize $H(Y)$.

First, play blind: you must choose your string knowing nothing about X. How well can you do? Consider submitting a fixed string a. If you choose $a = \texttt{00000}$, then matching produces output `00000` and failing to match reproduces X, which (conditional on not matching) is uniform over the 31 strings other than `00000`; a short calculation shows the output is then exactly uniform over all 32 strings: $H(Y) = 5$ bits, no reduction at all. Choosing $a = \texttt{00000}$ is the one genuinely wasted move.

Now choose any other fixed string, say $a = \texttt{11111}$. Two initial states lead to the output `00000`: the state $X = \texttt{11111}$ (you matched, and the function outputs zeros) and the state $X = \texttt{00000}$ (you did not match, and the function reproduces it). The output `11111` itself never occurs (if $X = \texttt{11111}$ you matched it, producing zeros; otherwise the output is $X \neq \texttt{11111}$). Every other string y occurs exactly when $X = y$. The output distribution is therefore

$$P(Y = \texttt{00000}) = \tfrac{2}{32} = \tfrac{1}{16}, \qquad P(Y = \texttt{11111}) = 0, \qquad P(Y = y) = \tfrac{1}{32} \;\text{ for the other 30 strings},$$

with entropy

$$H(Y) \;=\; \tfrac{1}{16}\log 16 \;+\; 30 \cdot \tfrac{1}{32}\log 32 \;=\; 0.25 + 4.6875 \;\approx\; 4.94 \text{ bits}.$$

A modest improvement over 5 bits, but a real one: you have piled two initial states onto a single output, making the output distribution slightly lumpy and therefore slightly more predictable. By symmetry every fixed non-zero string performs identically, randomizing among them does no better, and one can check this is optimal among blind policies; for this game and this initial distribution, the blind optimum is $\Delta H \approx 0.06$ bits. (If the initial distribution over strings were non-uniform, the best blind strategy would be to guess the most probable string that is not `00000`; the theorem's baseline $\Delta H^{\max}_{\text{blind}}$ takes the maximum over initial distributions as well.)

6.5 The same game, played sighted

Now suppose that before you choose your string, you are shown the first k bits of the computer's secret string. A natural strategy is to submit the string consisting of the k bits you saw, followed by all `1`s (the continuation does not matter much, so long as you avoid completing the all-zeros string). Seeing k bits multiplies your chance of an exact match by 2^k, from $\frac{1}{32}$ to $\frac{1}{2^{5-k}}$, and each match funnels probability onto the single output `00000`. Computing $H(Y)$ for each k exactly as above produces the following table.

bits of X observed	0	1	2	3	4	5
$H(Y)$ in bits	4.94	4.85	4.63	4.11	2.83	0
ΔH in bits	0.06	0.15	0.37	0.89	2.17	5

With all five bits observed you match the secret string every time, the output is deterministically `00000`, and you achieve the maximum conceivable reduction $\Delta H = H(X) = 5$ bits. Each additional bit of observation buys additional steering power, and the more you know, the faster the returns in this particular game. Information about the environment converts into optimization of the environment.

6.6 Mutual information measures sightedness

To state the theorem we need to quantify "how sighted" a policy is, and the right quantity is exactly the mutual information of Definition 2.6. In the strategy above with $k = 2$, your action is determined by the two bits you observed (you always play "the two seen bits, then `111`"). Imagine someone who knows your strategy and observes only your *action*, say $A = \texttt{10111}$. They can immediately infer that the secret string

begins with `10`: the action reveals the observed bits. Before seeing your action their uncertainty about X was $H(X) = 5$ bits; after seeing it, $H(X \mid A) = 3$ bits; so $I(X; A) = H(X) - H(X \mid A) = 2$ bits, exactly the number of bits the policy "knows". Mutual information formalizes the intuitive notion of how many bits of the environment's state are reflected in the agent's behavior, and it does so without any reference to the agent's internals: it is a property of the joint statistics of state and action, estimable in principle by watching the system play many rounds.

6.7 The theorem

Theorem 6.2 (Touchette–Lloyd)**.** *Fix any dynamics $P(Y \mid X, A)$. Then for every initial distribution $P(X)$ and every policy $P(A \mid X)$,*

$$\Delta H \;\le\; \Delta H^{\max}_{\text{blind}} \;+\; I(X; A).$$

In words: the entropy reduction achieved by an arbitrary policy decomposes into at most two budgets, namely everything the dynamics could have been steered to do by a blind controller, plus one bit for every bit of mutual information between the action and the environment's initial state. The theorem holds with no assumptions about the policy's internals, no optimality requirements, and no restriction on the dynamics. In particular, *it makes no reversibility assumption*: it is a purely information-theoretic (data-processing) inequality, valid for arbitrary dynamics, reversible or not. This is consistent with the rest of the development rather than in tension with it. The second law of Section 4.4 needed reversibility (in the form of double stochasticity) to forbid *global* entropy reduction; the Touchette–Lloyd theorem needs no such assumption to bound the entropy reduction of a steered subsystem *beyond the blind baseline*. The two results constrain different quantities, and both hold at once.

The contrapositive direction gives the theorem its selection-theorem flavor. Suppose we observe a system achieving an entropy reduction ΔH strictly exceeding the blind baseline of its environment's dynamics. Then we may *deduce*, sight unseen, that the system's actions carry at least $\Delta H - \Delta H^{\max}_{\text{blind}}$ bits of mutual information with the environment's state. Mutual information with the environment is plausibly a necessary (though certainly not sufficient) ingredient of anything deserving the name *world model*; the theorem is thus a first rigorous step along the path "observed optimization implies internal modeling", which is the path toward understanding when optimization implies agent-like structure. In the vocabulary of the guessing game: anyone who beats 4.94 bits of output entropy *must have peeked*, and the margin by which they beat it lower-bounds how much they saw.

6.8 What the theorem does not say

The inequality is an upper bound on what information makes possible, not a guarantee that information will be used well, and the gap matters in both directions.

First, mutual information can simply fail to help. If the dynamics ignore the action entirely (Y depends only on X), then every policy, however well-informed, performs exactly like a blind one: the channel from action to environment has zero capacity, and knowledge without influence is sterile.

Second, mutual information can be actively squandered. In the guessing game, consider the policy that observes the secret string completely and submits its bitwise negation (if the computer picked `01100`, play `10011`). This policy has the maximal $I(X; A) = 5$ bits of mutual information with the environment, and it *never* matches the secret string, so the output is always $Y = X$, uniform: $\Delta H = 0$, strictly worse than the optimal blind policy. Modeling the world is compatible with steering it badly.

So the theorem should be read as a conservation-style constraint, in the same family as the second law: it tells you what cannot happen (large optimization without modeling), never what must happen (modeling producing optimization). The analogy is exact in spirit, and Section 8 will turn it into a literal theorem of thermodynamics, where the role of $I(X; A)$ is played by the mutual information between a measurement record and the measured system.

Example 6.3 (Noise-cancelling headphones)**.** An everyday system displays the whole structure of the theorem. Let X be the ambient sound arriving at your ears, Y the sound you actually hear, and consider two technologies. Foam earplugs attenuate sound by passive damping. They are utterly blind ($I(X; A) = 0$; the plug's "action" is the same whatever the sound), and yet they achieve a substantial entropy reduction: they

exploit fixed statistical structure of the environment (sound is vibration, foam damps vibration), which is exactly a blind policy living off the $\Delta H^{\max}_{\text{blind}}$ budget, like the coin engine living off the known bias difference. Active noise-cancelling headphones do something categorically different: they *listen* to the incoming sound and emit its inverted waveform. The emitted signal carries high mutual information with the ambient sound, and the entropy reduction correspondingly exceeds anything passive damping can achieve; the headphones can even steer selectively, cancelling the drone of an engine while passing a human voice. And playing music through the headphones is an entropy-*increasing* action, a reminder that agents are not obliged to minimize entropy; the theorem merely prices the steering they choose to do.

Remark 6.4 (The coin engine, revisited). The generalized heat engine of Appendix A and the Touchette–Lloyd theorem are two halves of one picture. The engine shows what blind policies can extract: everything in the $\Delta H^{\max}_{\text{blind}}$ budget, which is funded by pre-known statistical disequilibrium (the temperature difference between the pools). The theorem prices what blindness cannot reach: every further bit of entropy reduction costs a bit of mutual information acquired by observation. This is the quantitative form of the Type-3 channel of Section 5: $I(X;A)$ is precisely the correlation an agent spends when it steers, and Section 8.6 will show, with the books balancing exactly, what acquiring it costs in turn.

7 The trouble with subjective entropy

Everything so far has treated the distribution μ, with respect to which all our entropies are defined, as given. This section examines that assumption and finds that it breaks down precisely at the point where agent foundations needs it to hold firm. The critique, and its resolution in the next section, follow Daniel C and Ebtekar's essay and the foundational paper of Ebtekar and Hutter.

7.1 Entropy is supposed to measure the capacity to optimize

Begin with what entropy is *for*. Across physics and engineering, entropy's job is to quantify a system's capacity to do work, and the developments so far (Sections 4–5, together with the coin world of Appendix A) let us read "capacity to do work" as *capacity to perform optimization*: negentropy is what fuels the funneling of broad distributions into narrow ones.

Apply this to the demon after a long stretch of Type-2 optimization. The demon's memory comprises, say, m bits, of which some portion is now filled with accumulated measurement records. The demon's *remaining* capacity to optimize the gas is its free memory: the total memory size minus the entropy already stored in it. Each further bit of optimization will consume another bit of free memory. Now, here is the question on which everything turns: *is the demon's free memory an objective, physical quantity?* It certainly ought to be. Whether the demon's hardware can absorb another billion measurements is a question about the demon's physical configuration, with engineering consequences (how much longer the refrigeration runs, how much heat will eventually be paid to clear the records). It would be strange for the demon's actual ability to perform useful work on the gas to depend on the beliefs of some external observer describing the demon. Yet that is exactly what the standard formalism delivers.

7.2 The exogenous distribution

Recall from Definition 2.4 the structure of the standard definitions. The entropy of an *individual* state x is undefined; what is defined is the Shannon codelength $\hat{H}(x,\mu) = \log \frac{1}{\mu(x)}$ of a state *relative to a distribution* μ, and the Gibbs–Shannon entropy $H(\mu)$, which is its average. The distribution μ is an input to the formalism, not an output: it represents somebody's probabilistic knowledge of the system, and the formalism itself is silent about whose knowledge it is, where it is stored, how it was obtained, or what it costs to have it. In the language of these notes, the standard framework treats knowledge as *exogenous*.

For the great equilibrium successes of thermodynamics, this exogeneity is harmless, because there is a canonical and effectively forced choice of μ. But notice what is required for the choice to be harmless. Call μ a *useful summary* of an actual physical state x when three conditions hold:

1. **Simplicity:** μ has a short description, such as "an ideal gas of N molecules at temperature 300 K and pressure 1 atm, at equilibrium". (Otherwise, specifying μ itself smuggles in unaccounted-for information; hold this thought.)

2. **Concentration:** the codelength concentrates near its mean, $\hat{H}(X, \mu) \approx H(\mu)$ with high probability under $X \sim \mu$. For systems composed of many weakly interacting parts, this follows from the law of large numbers; it is what lets a single number, "the entropy", stand in for a whole distribution of codelengths.

3. **Typicality:** the actual state x is an unremarkable sample from μ, falling in the high-probability set where the previous two conditions bite.

When all three conditions hold, $H(\mu) \approx \hat{H}(x, \mu)$, every reasonable notion of entropy agrees, and one may do thermodynamics with macrovariables alone, never asking where μ lives. Equilibrium statistical mechanics resides entirely inside this regime, and the classical apparatus of Clausius and Boltzmann (temperature, pressure, chemical composition as the privileged macrovariables) is well adapted to it.

7.3 Three ways the ensemble picture fails

Far from equilibrium, and above all for *information-bearing* states, each of the three conditions can fail independently, and each failure breaks the formalism in a different instructive way.

Failure of simplicity: knowledge with no ledger. Let μ be a point mass on one particular, intricate configuration x ("I know the exact microstate"). Then $H(\mu) = 0$, and the formalism declares the system fully known, hence fully available for work extraction: knowing the exact state of a gas, one can in principle extract all of its energy as work, using a transformation tailored to that state. But the tailored transformation, and the knowledge it embodies, had to be specified by *something*; the description of μ is exactly as intricate as x itself, and the formalism books it at zero cost. The Gibbs–Shannon account permits arbitrarily detailed knowledge as a free exogenous resource. Physically, any agent in possession of such knowledge must encode it in a memory, and acquiring, storing, and clearing that memory have costs that the second law cares about, as the demon already showed us. The ensemble formalism has no slot for these costs.

Failure of concentration: averages that describe no state. A robot flips a hidden fair coin and, depending on the outcome, either fully drains its battery or leaves it fully charged. Under the resulting fifty-fifty mixture μ, the Gibbs–Shannon entropy of the battery-plus-surroundings sits exactly halfway between the entropy of the charged state and that of the drained state, and a free-energy calculation based on $H(\mu)$ concludes that the robot can perform about half a charge of work. This is false in every possible world: the robot can perform either a full charge of work or none at all. The mean $H(\mu)$ is an average over an ensemble whose members behave nothing like the average, because the distribution is bimodal rather than concentrated. The number we actually want is a property of the *individual* state the robot is in fact in, and the ensemble formalism does not define such a number.

Failure of typicality: structure the ensemble cannot see. Suppose the actual state x, while formally a possible sample from the equilibrium ensemble μ, happens to be special in a way the macrovariables do not capture: the gas particles, say, momentarily occupy a strongly patterned, compressible configuration. The codelength $\hat{H}(x, \mu)$ prices x as if it were generic, and a work calculation based on it will *underestimate* what a sufficiently clever machine (one that recognizes and exploits the pattern, that is, a machine running a compression algorithm) can extract from x. Taken at face value, the ensemble calculation would brand such a machine a violator of the second law. The correct response is not to outlaw clever machines but to admit that the entropy relevant to what can be done with x is a property of x's actual structure, not of the ensemble we happened to embed it in.

7.4 The demon's capacity, and why this is the central case for agent foundations

Return now to the demon's free memory, and the puzzle lands precisely. The demon's remaining optimization capacity is m minus the entropy of its memory contents, but the entropy of its memory contents *relative to*

which distribution? If we, the observers, refine our beliefs about what the demon has recorded, the standard formalism says the entropy of the demon's memory drops, and the demon's "capacity to do work" rises, though nothing about the demon's hardware has changed. The classical macrovariable repertoire is no help either: temperature, pressure, and chemical composition are designed for systems whose unresolved degrees of freedom mix rapidly and forgettably, whereas a memory is, by engineering design, a system whose states are stable, distinguishable, and meaningful. Every distinct memory configuration must be treated as its own condition; there is no thermalizing ensemble over memory states for μ to latch onto, and there is no privileged ensemble over outputs of a long computation, because the whole point of computation is to produce states whose structure no simple prior anticipates.

For agent foundations, these are not pathological corner cases to be quarantined. They are the entire subject matter. The states we care about (an agent's memory and world model, the records left by measurements, computations in progress, an environment halfway through being reshaped toward a target) are exactly the far-from-equilibrium, information-bearing, intricately structured states on which the ensemble formalism fails. If entropy is to serve as the measure of optimization capacity for embedded agents, we need a definition of entropy that applies to an *individual physical state*, with no exogenous distribution anywhere in sight. Remarkably, such a definition exists, and it comes from the theory of computation.

8 Algorithmic thermodynamics

Algorithmic thermodynamics, developed by Ebtekar and Hutter on foundations laid by Bennett, Zurek, Gács, and Levin, keeps the entire coarse-grained Markov framework of Section 4 and changes exactly one thing: it replaces the subjective ensemble μ with universal computation. Instead of asking "how surprising is the state x under the distribution μ?", it asks "how hard is x to describe, full stop?" The answer turns out to support a complete thermodynamics: a second law, fluctuation bounds, a Landauer principle, and an exact treatment of measurement and feedback, all defined for individual physical states.

8.1 Kolmogorov complexity

Fix a universal computer U; concretely, think of an interpreter for your favorite general-purpose programming language. Every coarse-grained state x of a physical system is identified, via the coarse-graining's encoding, with a finite binary string, so it makes sense to talk about programs that output x.

Definition 8.1 (Description complexity)**.** The *Kolmogorov complexity* (or description complexity) $K(x)$ of a string x is the length, in bits, of the shortest program that outputs x when run on U. The *conditional complexity* $K(x \mid y)$ is the length of the shortest program that outputs x when given y as auxiliary input. The *algorithmic mutual information* between two strings is

$$I(x : y) \;:=\; K(x) + K(y) - K(x, y),$$

the number of bits that a description of one saves in describing the other.[3]

The right mental model for $K(x)$ is *optimal lossless compression*: $K(x)$ is the size of the most compressed self-contained representation from which x can be exactly regenerated. A few examples calibrate it. The string consisting of a million zeros has tiny complexity: a fifteen-byte program prints it. The first million digits of π have tiny complexity too, despite passing every statistical test for randomness: a short program computes them. A million bits fresh from a quantum random number generator have complexity close to a million bits: with overwhelming probability there is no structure to exploit, and the shortest "program" is essentially a verbatim quotation of the string. A physical state of 10^{23} particles all sitting in one corner of a box has low complexity (a short loop prints the repeated coordinates), while a generic thermalized state

[3]Two pieces of technical fine print, recorded once and then suppressed. First, programs are required to be *self-delimiting*: no valid program is a prefix of another, so a program announces its own end. This makes program lengths behave like codeword lengths; in particular they satisfy the Kraft inequality $\sum_x 2^{-K(x)} \le 1$, which is what lets $2^{-K(x)}$ act like a (sub)probability distribution. Second, the chain rule for K holds in the form $K(x, y) \stackrel{+}{=} K(y) + K(x \mid y, K(y))$, with the complexity of y itself appearing in the conditioning; consequently some identities below, including the second expression for $I(x : y)$, implicitly carry such terms. Every statement we make is correct with these refinements installed, and none of them matter at physical scales.

has complexity close to the full length of its encoding. The slogan: *low entropy is compressibility*, and it is a property of the individual state x, with no ensemble in sight.

Algorithmic mutual information behaves the way Section 2's mutual information does, but for individual objects: $I(x : x) \stackrel{+}{=} K(x)$ (a copy of x saves you everything), $I(x : y) \stackrel{+}{=} 0$ for typical independent random strings (a description of one is useless for the other), and intermediate values measure partial correlation, such as between a measurement record and the system it measured.

Because the shortest program may be arbitrarily hard to find, statements about K hold up to additive constants reflecting fixed wrapper code; following Ebtekar and Hutter we write $f \stackrel{+}{<} g$ for $f \le g + c$, $f \stackrel{+}{>} g$ for $f \ge g - c$, and $f \stackrel{+}{=} g$ for both, where c depends only on fixed choices like the universal computer, never on the strings involved.

8.2 Why K deserves to be called entropy

Three families of facts justify treating $K(x)$ as *the* entropy of the individual state x.

The choice of computer is physically negligible. K depends on the universal computer U, which looks alarming for a proposed physical quantity. But the dependence is bounded by translation: for any two universal computers, a fixed interpreter converts programs for one into programs for the other, so the two versions of K differ by at most the interpreter's length, uniformly in x. To gauge the scale, convert to physical units: entropy in bits converts to thermodynamic entropy at 1 bit $= k_{\mathrm{B}} \ln 2 \approx 9.57 \times 10^{-24}$ joules per kelvin. Suppose two programming languages were so alien to each other that translating between them required a 12-gigabyte interpreter, far larger than any real interpreter. Twelve gigabytes is about 10^{11} bits, so the two languages would assign every physical system entropies agreeing to within 10^{-12} J/K, far below experimental resolution for macroscopic systems. For all physical purposes, K is computer-independent.

K agrees with the Shannon picture exactly where the Shannon picture is valid. Let μ be any computable probability distribution. Then for *every* state x,

$$K(x \mid \mu) \stackrel{+}{<} \log \frac{1}{\mu(x)} = \hat{H}(x, \mu),$$

because one way to describe x, given access to μ, is to transmit its Shannon codeword (Section 2.3); the shortest description can only be shorter. In the reverse direction, a counting argument based on the Kraft inequality shows that the inequality is nearly tight for nearly all samples: for any $\delta > 0$, with probability at least $1 - \delta$ over $X \sim \mu$,

$$K(X \mid \mu) \stackrel{+}{>} \log \frac{1}{\mu(X)} - \log \frac{1}{\delta}.$$

In words: the optimal universal description of a typical sample is its Shannon codeword, up to logarithmically small slack; atypical samples can beat the Shannon codeword, but only a δ-fraction can beat it by more than $\log \frac{1}{\delta}$ bits. Averaging the two displays recovers Zurek's identity

$$H(\mu) \stackrel{+}{=} \langle K(X \mid \mu) \rangle_{X \sim \mu} :$$

the Gibbs–Shannon entropy is the mean Kolmogorov complexity of a sample, given prior knowledge of the distribution. Two further specializations are worth knowing. Taking μ uniform on a finite set B shows that for any simply describable set B containing x, we have $K(x) \stackrel{+}{<} K(B) + \log |B|$, with near-equality for the vast majority of elements; taking B to be a Boltzmann macrostate (the set of all microstates sharing given macrovariable values) recovers the Boltzmann entropy $\log |B|$ as an upper bound on K, achieved by typical members. And the bound applies to *any* simple set, not just classical macrostates: Ebtekar and Hutter's example is that the entropy of a bookshelf can be estimated by taking B to be the set of configurations compatible with how the books are sorted. The algorithmic entropy automatically considers every simple description of the state, classical macrovariables included, and charges the state the cheapest one.

These facts settle the three failure modes of Section 7.3. Where the ensemble picture is valid (simple μ, concentration, typical x), $K(x) \approx \hat{H}(x, \mu) \approx H(\mu)$, and algorithmic entropy reproduces the classical answers;

nothing is lost. Where the ensemble picture fails, K keeps working: a point-mass μ on an intricate state no longer yields zero entropy, because $K(x)$ is large regardless of what distribution we mention alongside it, so the "free knowledge" loophole closes (the knowledge is now priced, in the only currency that matters, description length); the robot's battery gets a definite per-state answer, K(charged state) or K(drained state), with the ensemble value revealed as a mere average of the two by Zurek's identity; and a secretly patterned gas configuration gets credited for its structure, $K(x) \ll \hat{H}(x, \mu)$, so the clever compression machine extracts exactly the work that x's actual description length permits, violating nothing.

K is uncomputable, and a second law requires this. No algorithm computes $K(x)$ from x. One direction of approximation is available: by running ever more candidate programs for ever longer, one obtains a decreasing sequence of upper bounds converging to $K(x)$ (in the jargon, K is upper semicomputable), which is why real-world compressors give genuine upper bounds on entropy. But no algorithm can certify large *lower* bounds on complexity: there is no procedure that, given x, verifiably reports "$K(x)$ is at least one million". (This is Chaitin's incompleteness theorem, and the proof is one line of self-reference: if such a certifier existed, then a short program could enumerate strings until it finds one certified to have complexity at least a million and print it, thereby describing a supposedly million-bit-complex string with a few hundred bits, a contradiction.)

At first sight uncomputability looks like a defect in a proposed physical quantity. It is in fact load-bearing, by an argument worth internalizing. Suppose we tried to do thermodynamics with any entropy-like state function K' for which an algorithm *could* certify large values. Then a short program p could search for and output a state x^* certified to satisfy $K'(x^*) \gg 0$. But any short program that can *construct* x^* can also *erase* it: run p to produce a second copy of x^* alongside the existing one, use the copy to reversibly cancel the original (a controlled-NOT for every bit, exactly as in Example 5.1), then run p backward to clean up. The net effect of a constant-size machine is to take the world from a state containing x^* (with K'-entropy $\gg 0$) to a state not containing it (K'-entropy ≈ 0). A small, simple, reversible machine that can destroy unbounded amounts of "entropy" at will is a perpetual motion machine with respect to K': no second law can hold for any computable notion of entropy. The uncomputability of K is precisely what closes this loophole; it guarantees that no simple machine can systematically recognize which states are simple-but-disguised, and therefore none can systematically un-disguise them. (Note the parallel with Section 6.8: knowledge that cannot be obtained cannot be spent.)

8.3 The algorithmic second law

Return to the physical setting of Section 4: a coarse-grained state space with equal-volume cells, evolving as a Markov chain with doubly stochastic, *computable* transition probabilities (the laws of physics are assumed to have a short program, a complexity-theoretic version of the Church–Turing thesis; we choose our reference computer so that the dynamics are simply describable). In this setting the algorithmic entropy of the system in state x is just $K(x)$, and it obeys a second law, which descends from Levin's law of *randomness conservation*.

Theorem 8.2 (Algorithmic second law; Levin, Ebtekar–Hutter, stated informally)**.** *Let (X_t) be a computable, doubly stochastic Markov chain modeling an isolated system's coarse-grained evolution, and let $s < t$ be two times. Then for every $\delta > 0$, with probability greater than $1 - \delta$,*

$$K(X_s) - K(X_t) \overset{+}{<} K(t - s) + \log \tfrac{1}{\delta}.$$

That is: *the description complexity of an isolated system's state almost never decreases, except by a precisely priced fluctuation allowance.* The theorem improves on the ensemble second law (Theorem 4.1) in two ways that matter for us. It holds for each individual trajectory with high probability, not merely for the average of an ensemble; and it requires no initial distribution μ at all, removing the exogenous ensemble exactly as Section 7 required.

The two terms of the allowance are each interpretable, and each is very small in physical terms.

The term $\log \frac{1}{\delta}$ prices *chance* fluctuations: rare trajectories on which entropy temporarily falls. Its scale is set by how rare a fluctuation you care to consider. Tolerating events of probability $\delta = 2^{-1000}$, the allowed entropy dip is about a thousand bits, which in physical units (recall 1 bit $\approx 10^{-23}$ J/K) is around 10^{-20} J/K:

twenty orders of magnitude below anything measurable. The statistical character of the second law is real, but quantitatively trivial at macroscopic scales.

The term $K(t-s)$, the complexity of the *elapsed time*, prices *deterministic recurrences*, and its necessity is a subtle point worth spelling out. The Poincaré recurrence theorem guarantees that an isolated finite system eventually returns arbitrarily close to its initial low-entropy state, so entropy cannot literally always increase. How does the theorem survive? Because recurrence times are astronomically long and arithmetically structureless: a system whose recurrence happens at step $t - s = 17{,}382, \ldots$ (imagine 10^{20} digits) revisits low entropy only at moments whose timestamps are themselves about as complex as the state, and the $K(t-s)$ allowance covers exactly this. For any time interval you can *simply specify* (10^9 years, 2^{100} Planck times), $K(t-s)$ is a few hundred bits at most, and entropy, regarded as a function of simply specified times, increases monotonically up to negligible slack.

One special case is worth stating separately, since the demon analysis below uses it repeatedly.

Corollary 8.3 (Deterministic dynamics cannot change entropy)**.** *If the evolution over the interval is a computable bijection f with a short description (for instance, a few steps of simply describable reversible dynamics), then*

$$K(f(x)) \stackrel{+}{=} K(x).$$

The reason is immediate from the compression picture: given the dynamics, a description of x is a description of $f(x)$ and vice versa, so the two complexities differ by at most the (small) complexity of f itself. The corollary has an important consequence: *substantial entropy production requires randomness.* A deterministic, simply describable evolution can shuffle states but cannot make them more complex by more than a constant; in classical physics, the randomness that drives entropy production is supplied by coarse-graining chaotic microdynamics (Section 4.3), which continually injects effectively fresh random bits into the coarse description. A useful way to picture it: a doubly stochastic evolution is equivalent to drawing a random bijection at each step (say, by tossing coins), and entropy can increase by at most the number of coins tossed, and only to the extent that the coins are independent of the state. The Shannon-level second law is blind to this distinction between deterministic reshuffling and genuine randomization; the algorithmic version sees it.

8.4 Heat, temperature, and Landauer's principle

So far our systems have been isolated. To connect with real thermodynamics (and to pay off several debts incurred above) we now couple the system of interest to a *heat reservoir* and meet the only two pieces of genuinely physical apparatus in these notes: temperature, and the bit-to-joule exchange rate.

A heat reservoir is a large, rapidly mixing system (a tank of water, the ambient air) described only by its total energy E_{res}. Its defining property is an enormous degeneracy: for each value of the energy there is some number of microstates consistent with it, and we write $B(E_{\text{res}})$ for the logarithm of that number, the reservoir's *Boltzmann entropy* in bits, which grows with energy (more energy, more ways to distribute it).[4] The reservoir's *temperature* encodes the exchange rate between energy and entropy at the margin:

$$\frac{1}{T} \;\propto\; \frac{\partial B}{\partial E_{\text{res}}}, \qquad \text{normalized so that delivering } k_{\text{B}}T\ln 2 \text{ joules of heat raises } B \text{ by exactly one bit.}$$

A hot reservoir (large T) gains little entropy per joule; a cold one gains a lot. This single definition is the entire bridge between information and energy: *a bit of entropy, dumped into a reservoir at temperature T, costs $k_{\text{B}}T\ln 2$ joules of energy, and conversely.* At room temperature the rate is about 3×10^{-21} joules per bit.

Now consider a transition $x \to y$ of our system while it emits heat Q into the reservoir (negative Q means absorption). The *total* algorithmic entropy change has two parts: the system's, $\Delta K := K(y) - K(x)$, and the reservoir's, $Q/(k_{\text{B}}T\ln 2)$ bits. Applying the algorithmic second law (Theorem 8.2) to the joint system

[4] In the coarse-graining language: reservoir cells are indexed by energy and have wildly unequal volumes, which is exactly the situation that the footnote at the end of Section 4.3 promised we would eventually need. The general Gács entropy of a joint state is $K(\text{system state}) + \log(\text{volume of the reservoir's cell})$, and what follows is that formula in words.

yields, with probability greater than $1-\delta$,

$$\Delta K \;+\; \frac{Q}{k_{\mathrm{B}}T\ln 2} \;\overset{+}{>}\; -\log\tfrac{1}{\delta}. \tag{1}$$

This single inequality is the algorithmic form of *Landauer's principle*. Read it in the direction of erasure: if a memory's contents are simplified, $\Delta K < 0$ (clearing data means reducing the description complexity of the memory's physical state), then the heat term must make up the difference, so erasing one bit of complexity forces the emission of at least $k_{\mathrm{B}}T\ln 2$ joules of heat, up to the usual negligible fluctuation allowance. Erasure is not free; it is the moment when Type-2 bookkeeping (entropy parked in memory) is converted into Type-1 bookkeeping (entropy exported as heat). The algorithmic formulation adds something the classical one lacks: an unambiguous, observer-free criterion for what counts as erasure. What must be paid for is the reduction of the memory state's *description complexity*, not the reduction of any particular observer's uncertainty.

Two refinements, both from Ebtekar and Hutter, correct widespread misreadings of Landauer's principle, and both follow from splitting the heat into the two terms of (1). The quantity $-\Delta K$ (entropy decrease of the system) is called the *Landauer cost* of the transition, and the slack in the inequality (total entropy produced, system plus reservoir) is the *entropy production* or *EP cost*; the heat emitted is their sum.

First, *logical irreversibility does not by itself imply heat emission.* Consider repeatedly overwriting a register with fresh uniform random values. Each overwrite is logically irreversible (the old value is unrecoverable), yet the register's complexity stays pinned near its maximum, so $\Delta K \approx 0$: the Landauer cost is zero, the dynamics are doubly stochastic and need no reservoir contact at all, and no heat flows. What costs heat is not destroying *logical* recoverability but destroying *complexity*.

Second, *Landauer costs are reversible and cancel over time; only entropy production is a permanent expense.* A computer's memory has bounded size, so over a long run its complexity cannot trend: $\Delta K \approx 0$ in the long-time average, positive and negative Landauer costs cancelling. By (1), the long-run heat emission, and hence the energy bill of computation, equals the accumulated *EP cost*, the entropy genuinely produced by irreversible operation, not the Landauer cost of individual erasures. A perfectly reversible computer could, in principle, run with vanishing energy expenditure, paying Landauer costs and earning them back in equal measure.

8.5 The thermodynamic costs of information processing

With (1) in hand, we can audit the three elementary information processes an embedded agent performs (randomization, computation, and measurement) and locate exactly where each one does, and does not, incur an irreducible cost. The unifying principle: *the absence of entropy is a resource to be managed, and every process that handles information either preserves the resource, converts it between forms, or wastes it as entropy production.*

Randomization. Suppose an agent needs random bits (say, as a seed). Generating them raises the memory's complexity: $\Delta K > 0$, a *negative* Landauer cost. Done carefully, the entropy can be *drawn from* a heat reservoir, cooling the reservoir by $k_{\mathrm{B}}T\ln 2$ per random bit acquired, with zero entropy production; later, returning the entropy (clearing the seed) warms the reservoir back, and the full cycle has zero net heat flow. Done carelessly (the memory becomes random through uncontrolled interaction, without cooling anything), the negative Landauer cost is balanced by positive EP instead, and the eventual cleanup emits net heat. Randomness is free if harvested reversibly, costly if scavenged.

Computation. Let a deterministic program generate a long output string x (the digits of π, a pseudorandom sequence, a rendered scene). By Corollary 8.3, the output's complexity is bounded by the program's: $K(x)$ is small however long and disordered x *looks*. The honest way to dispose of such a string is to *uncompute* it, running the generating program backward at no thermodynamic cost. The dishonest way is to forget its provenance and dump it into the environment as if it were random: since the string was never complex, $\Delta K \approx 0$ and nothing was erased, so the resulting heat is pure entropy production, an avoidable waste caused by a mismatch between the data's actual structure and the machinery handling it. Pseudo-entropy becomes real entropy precisely when it thermalizes, that is, when its compressible structure is destroyed by mixing;

afterward, not even a perfect compressor can reclaim it. (Hold on to this thermalization point; it returns in Section 9.)

Measurement. Measurement is reversible copying, as we saw with the demon, and a measurement can be *undone* at zero cost by a second interaction with the same object (uncopying against the source). The irreducible cost appears only when the correlation is wasted: if either copy is lost *without* the two interacting again (the memory is overwritten, or the measured system changes state on its own), the mutual information between them is destroyed. For two non-interacting systems x and y, each of $K(x)$, $K(y)$ can only increase (each is isolated, so Theorem 8.2 applies separately) and $I(x : y)$ can only decrease (correlations cannot grow without interaction), so in the decomposition of the total entropy

$$K(x, y) \;=\; K(x) + K(y) - I(x : y),$$

all three contributions to entropy production are nonnegative, and *discarded mutual information is itself a form of entropy production.* Knowledge about a system that has moved on is fuel that has evaporated.

In summary: negentropy can be exchanged with a reservoir (randomization), hidden inside a computation's provenance (computation), or stored as correlation between systems (measurement). All three manipulations can in principle be performed reversibly, at zero net cost; every actual cost is an EP cost, incurred at the point where the machinery mismatches the information it handles.

8.6 Maxwell's demon, exactly

We now redo the demon, this time as theorem rather than story, and recover the Touchette–Lloyd bound as a statement about individual physical states. Let the demon's memory and the gas be jointly coarse-grained, writing joint states as pairs (memory contents, gas state). The demon's memory starts in a blank reference state 0; the gas starts in some state x.

Full measurement, then erasure. In the idealized case the demon performs a complete measurement, reversibly copying the gas's state into memory:

$$(0, x) \;\mapsto\; (x, x),$$

and then, using its copy as the control of a conditional operation (a string of controlled-NOTs, Example 5.1), reversibly resets the gas to a reference state:

$$(x, x) \;\mapsto\; (x, 0).$$

Each map is injective on the states that can actually occur (the first on pairs with blank memory, the second on pairs whose two slots agree), hence extendable to a bijection of the joint state space, and each is simply describable. Corollary 8.3 therefore applies to both steps, and the *total* complexity never moves:

$$K(0, x) \;\stackrel{+}{=}\; K(x, x) \;\stackrel{+}{=}\; K(x, 0) \;\stackrel{+}{=}\; K(x).$$

Look at what these constants say. The gas has gone from complexity $K(x)$ to complexity $\stackrel{+}{=} 0$: a complete Type-2 optimization. The joint complexity never changed: the second law was never threatened, at any step, without any need to "complete a cycle" or invoke ad hoc accounting. And the balance is now carried entirely by the demon's memory, which holds a record of complexity $K(x)$. The gas's erasure was permitted *precisely because a copy existed*; formally, "clear slot two given that slot one holds a copy" is injective. The step that the second law forbids is clearing the *last* copy:

$$(x, 0) \;\mapsto\; (0, 0)$$

is either not injective (if it is to work for every x) or not simply describable (if hard-wired for one particular complex x, the wiring itself would have complexity $K(x)$, and Corollary 8.3 charges for the dynamics' complexity). To get its memory back, the demon must pay the Landauer bill (1): $K(x)$ bits flushed into a reservoir at temperature T cost $k_{\mathrm{B}}T \ln 2 \cdot K(x)$ joules of heat. Szilard's classic resolution of the demon paradox (the demon's information processing saves the second law) is here not an extra postulate but a two-line calculation.

Partial measurement: the algorithmic Touchette–Lloyd bound. Realistic demons measure only a little. Let the measurement be $m(x)$, where m is any simply describable (possibly random, possibly many-to-one) function of the gas state: a few bits about one approaching molecule, say. The demon records the measurement, then uses the record as a control to drive the gas from x to some new state y:

$$(0, x) \;\mapsto\; (m(x), x) \;\mapsto\; (m(x), y).$$

Whatever clever feedback the second step implements, as long as it is a simply describable mixture of reversible operations, the algorithmic second law applied to the *joint* system gives

$$K\big(m(x), x\big) \stackrel{+}{<} K\big(m(x), y\big).$$

Expand both sides with the chain rule ($K(m, \cdot) \stackrel{+}{=} K(m) + K(\cdot \mid m)$, fine print as in Definition 8.1), subtract $K(m(x))$ from both sides, and rewrite conditional complexities via mutual information ($K(x \mid m) \stackrel{+}{=} K(x) - I(m : x)$):

$$K(x) - I\big(m(x) : x\big) \stackrel{+}{<} K(y) - I\big(m(x) : y\big) \;\leq\; K(y),$$

where the last step uses nonnegativity of algorithmic mutual information. Rearranged:

$$\boxed{K(x) - K(y) \stackrel{+}{<} I\big(m(x) : x\big).} \tag{2}$$

The gas can lose at most as much entropy as was measured from it. And the budget is exactly achievable: if the demon spends its correlation completely and reversibly (so that $K(m(x), x) \stackrel{+}{=} K(m(x), y)$, no entropy produced, and $I(m(x) : y) \stackrel{+}{=} 0$, no correlation left unspent), then the displayed inequalities collapse to

$$K(x) - K(y) \stackrel{+}{=} I\big(m(x) : x\big).$$

Now place (2) beside Theorem 6.2 and read the two term by term. In each, the left side is the entropy reduction of the steered system, and the right side is the mutual information between the controller's record (action, measurement) and the system's state. The Touchette–Lloyd theorem is the ensemble (Shannon) version of the constraint: it speaks of distributions, averages, and a blind baseline supplied by the dynamics. Inequality (2) is the single-shot, individual-state (algorithmic) version: it speaks of the one actual gas state in front of the demon, with no ensemble anywhere, and the role of the blind baseline is played by the additive constant (which absorbs what the simply describable dynamics can do unaided). It is, simultaneously, the rigorous form of Type-3 optimization from Section 5: the copy step purchases $I(m(x) : x)$, the control step spends it, bit for bit.

One last reading of the demon's situation will be the key to the final section. After the measurement, two distinct entropies attach to the same gas: its *objective* entropy $K(x)$, and its entropy *from the demon's perspective*, the conditional complexity

$$K\big(x \mid m(x)\big) \stackrel{+}{=} K(x) - I\big(m(x) : x\big),$$

which is smaller, by exactly the measured information. The demon's optimization power over the gas is precisely the wedge between the objective and the subjective entropy. Subjectivity has not been banished from thermodynamics; it has been *located*: the "subjective distribution" of the old formalism has become a physical conditioning on a physical record, carried in a memory that occupies space, obeys the second law, and pays Landauer rent.

8.7 An information engine: compressible strings as fuel

We close the section by closing a loop. Wentworth's coin engine (Appendix A) extracted work from *statistical* disequilibrium: the designer knew the pools' biases in advance, and that ensemble knowledge was the fuel. Ebtekar and Hutter's *information engine* is the same machine rebuilt on algorithmic foundations, and it runs on the compressibility of the individual strings it actually encounters, no ensemble required.

The engine inhabits an abstract universe of strings, with no energy and no reservoirs; its only resource is negentropy, which for a string z stored in m bits of memory is the compressibility $m - K(z)$, the gap between the space the string occupies and the space it needs. (Compare Appendix A.6: fuel is the gap between actual entropy and the constrained maximum. Here the constraint is memory size.) Concretely, the engine has an m-bit memory holding a string x in a self-delimiting format (a length header, then x, then padding), with the unused capacity held as a reserve of zeros, and it carries a fixed lossless compression algorithm f, with a bounded worst-case expansion of c bits on incompressible inputs. The engine cycles through three strokes:

1. **Burn.** Spend zeros from the reserve to perform useful tasks (next paragraph), in the process replacing some of the padding with waste data y.
2. **Eat.** With one reversible swap, expel the waste into the environment and take in a fresh string z from some promising location.
3. **Digest.** Apply the compressor: $z \mapsto f(z)$. If z was compressible, the memory now holds a shorter string and the zero reserve has grown; the engine has refueled.

What are zeros good for? They are *ancilla bits*: blank space that lets many-to-one operations be embedded inside one-to-one operations. Every irreversible act an agent might want to perform (overwriting a target with desired data, error correction, healing, repair, all of which map many "bad" configurations onto few "good" ones) can be implemented reversibly by swapping the displaced information into blank padding instead of destroying it. The demon's blank memory was exactly such an ancilla: the transition $(0, x) \mapsto (m(x), y)$ is lawful where the bare $x \mapsto y$ is not. A vivid biological reading: an organism that copies its genome over a patch of environment performs an irreversible overwrite; implemented reversibly, the organism absorbs the displaced environmental data into its own padding, then periodically eats something compressible to regenerate the padding. In Ebtekar and Hutter's worked illustration, an engine with 120 bits of capacity burns its zero reserve to clear a target region, reproduces its genome there, swaps its accumulated incompressible waste for the compressible environmental string `YummyAlphabetSoup`, and digests that string to restore its reserve, every step reversible, with the books balancing exactly.

The engine also fails informatively. If the eaten string turns out to be incompressible, digestion *costs* up to c zeros instead of yielding any (the compressor's worst-case expansion), and an engine that consistently eats noise runs its reserve down to nothing, at which point no further irreversible operations can be embedded and the engine, in their phrase, starves to death. Foraging for compressible structure is not optional; it is the energy economy. And notice the inversion that completes this section's arc: where the coin engine's designer needed to know the fuel's distribution in advance, the information engine needs no distributional assumptions at all. Its compressor either finds structure in the actual string before it or does not. Universal computation has replaced the exogenous ensemble even in the engine room.

9 Knowledge as a physical resource: optimization for embedded agents

The pieces are all on the table. This section assembles them into the synthesis that the agent foundations reading of this material has been driving toward.

9.1 From exogenous to endogenous knowledge

Set the two frameworks side by side one final time.

In the Gibbs–Shannon framework, what an agent knows about a system is represented by the distribution μ, and μ is exogenous: it lives in the modeler's ledger, outside the physics. Entropy, work capacity, and hence optimization capacity are all defined *relative to* this ledger entry. Change the ledger and the physics appears to change, which is the problem of Section 7.4; and the ledger itself is free, which is the unaccounted-knowledge loophole of Section 7.3. The framework works beautifully at equilibrium because there the ledger entry is forced and cheap; it breaks exactly when the knowledge gets interesting.

The lesson of embedded agency is that there is no such ledger. An embedded agent's probabilistic knowledge of its environment must be physically encoded in the agent's memory, which is made of the same matter

as the environment, occupies the same universe, and obeys the same laws. Algorithmic thermodynamics is the framework in which this lesson becomes load-bearing rather than merely admonitory. The agent's memory is a physical state a. The environment subsystem is a physical state s. And "what the agent knows about the environment" is, without any further modeling choices, the *algorithmic mutual information*

$$I(a : s) \stackrel{+}{=} K(s) - K(s \mid a),$$

the number of bits by which the agent's physical configuration shortens the description of the environment's. No distribution is mentioned. Knowledge has become an objective, physical relation between two pieces of matter, exactly as much a fact about the world as a distance or a voltage. Subjective probabilistic beliefs are not eliminated by this move; they are *implemented*. An agent whose memory encodes a good model of the environment is an agent whose physical state has high algorithmic mutual information with the environment's state, and the conditional complexity $K(s \mid a)$ is the environment's entropy *as that agent finds it*: the demon's-eye view, made rigorous at the end of Section 8.6.

And the results of these notes say, from three independent directions, that this quantity is a *resource*:

- By the algorithmic Touchette–Lloyd bound (2), $I(a : s)$ is an upper limit on how far the agent can reduce the environment's entropy beyond what simple blind dynamics achieve, and the limit is attainable: mutual information converts to optimization, bit for bit.
- By the three-types analysis (Section 5), the conversion is the Type-3 channel, and the channel is consumptive: spent correlation is gone, and the agent must re-measure (Type 2) or accept waste (Type 1) to continue.
- By the information-processing audit (Section 8.5), the resource is perishable: if the environment changes while the agent's records stand still, the mutual information decays, and discarded correlation is entropy production, fuel evaporated without work done.

The slogan form of the synthesis: *an embedded agent's knowledge about the world is the very same physical quantity as its capacity to optimize the world beyond blind baselines.* Agents that know more can do more, not as a psychological generalization but as a theorem of thermodynamics, with the exchange rate (one bit of steering per bit of correlation, $k_{\mathrm{B}}T \ln 2$ joules per bit flushed) fixed by physics.

9.2 The three types, sharpened by universal computation

Re-auditing the three optimization channels with algorithmic entropy in hand, each acquires a computational refinement, and in every case the refinement turns on the compressibility of an individual state, which the ensemble account cannot even express.

Type 1, refined. An agent about to dump waste into the environment should ask whether the waste is really as entropic as it looks. If the would-be waste carries compressible structure (and Section 8.5 showed that the outputs of computations always do), the agent can compress it before release, exporting $K(\text{waste})$ bits of true entropy rather than the waste's raw size. The compression must happen *before* the waste thermalizes into the environment, because thermalization destroys the compressible structure irreversibly; pseudo-entropy hardens into real entropy on contact with a reservoir.

Type 2, refined. The demon's memory bill for absorbing a gas state x is not the raw length of the measurement transcript but its complexity $K(x)$: records can be stored compressed, so a given memory can absorb more optimization than its nominal size suggests, by exactly the compressibility of what it observes. Dually, $K(x)$ is a floor: no ingenuity stores the state of x in fewer bits.

Type 3, refined. The steering budget is the algorithmic mutual information (2), and universal computation opens a remarkable acquisition route: if the environment's state is itself objectively simple ($K(s)$ small), then a short program generates it, and an agent can come to know it (acquire $I(a : s) \approx K(s)$, enough to

steer it all the way down) by *computation alone*, with hardly any measurement. Simple worlds are cheap to know and therefore cheap to optimize.

Threading the three refinements together reveals a tidy duality, observed by Daniel C and Ebtekar: the complexity $K(s)$ of the optimized subsystem is simultaneously the minimal memory required to optimize it by measurement (Type 2) and the minimal knowledge required to optimize it by correlation (Type 3). What a state costs to absorb and what it takes to know are the same number of bits, because both are, at bottom, the length of its shortest description.

9.3 Dissolving the subjectivity

We owe Section 7.4 an answer to its puzzle: does the demon's capacity to do work change when *our* beliefs about its memory change? The algorithmic framework answers in two steps, and the second step is the one that resolves the puzzle.

First, the demon's capacity is objective. Its remaining optimization capacity is its free memory, which algorithmic thermodynamics evaluates as the memory's size minus the *description complexity* of its current contents. If the contents are compressible, the demon can in principle compress them (a reversible, costless operation by Corollary 8.3) and recover the freed space; if the contents are incompressible, the occupied space is irreducibly occupied, because freeing it would require destroying information, which reversibility forbids and Landauer prices. Either way, the capacity is a function of the memory's physical configuration alone. Nobody's beliefs appear in the formula.

Second, the appearance of belief-dependence was tracking something real, and the resolution is to notice *whose* physics the changing beliefs belong to. When we, observing the demon, update our distribution over its memory contents, that update is not a disembodied shift in an abstract ledger: it is a physical change in *our own brains*, which now carry increased algorithmic mutual information with the demon's memory state. That mutual information is itself a Type-3 resource: in principle we could spend it to help compress the demon's memory, freeing capacity the demon alone could not access. So the demon's situation genuinely is different after our update, but only *relative to the joint system that now includes our correlated brains*, and the difference is exactly as large as the physical correlation we acquired. What the old formalism booked as a mysterious observer-dependence of entropy, the new formalism books as an ordinary physical relation between observer and observed. Every ledger is somebody's memory; once the ledgers are made of atoms, the books balance with no observer-shaped hole.

9.4 Back to optimizing systems

Close the loop with Section 3. An optimizing system, in Flint's sense, robustly funnels a broad basin of configurations into a narrow target set. The machinery of these notes now tells us what the existence of such a system entails, and the entailments land neatly on the three axes of Section 3.3.

The funneling itself is coarse-grained entropy reduction in a subsystem, so somewhere in the total system the books must balance through some mixture of the three channels: entropy exported to the surroundings (Type 1), records accumulating in some memory-like part (Type 2), or pre-existing correlation being consumed (Type 3). *Robustness*, the depth and width of the basin the system can keep funneling, is physically the size of the system's entropy-disposal capacity: its reservoir access, its free memory, its stock of correlation with what it steers. If the system's funneling exceeds the blind baseline of its environment's dynamics, then by Theorem 6.2 and its algorithmic sharpening (2), the system *must* contain mutual information with the subsystem it steers. Mutual information with the environment is a necessary (though, as Section 6.8 stressed, not sufficient) ingredient of anything we would call a world model, so this is a first thermodynamic step toward the selection-theorems program: at least this much modeling is forced by the bookkeeping rather than read into the system by an observer. *Duality*, the separability of engine from object, is the question of whether the bookkeeping is localized: a highly dualistic optimizer like the robot keeps its records, its correlations, and its waste stream in an identifiable place (which is also why it is brittle there, as Flint observed and as the demon's overwriteable memory makes precise). A non-dualistic optimizing system like the tree distributes the books through its whole fabric. And *retargetability* is the question of whether the target is held as a compact, rewritable record: a system steered by a short representation can be redirected

by flipping few bits precisely because, as the information engine taught us, a compact record is the cheap, spendable form of the knowledge that funds the funneling.

10 Takeaways

- Optimization, characterized descriptively, is the robust evolution of a system from a broad basin of attraction into a narrow target set (Flint's optimizing systems), and its quantitative measure is entropy reduction: the bits by which the process narrows the world's possibilities. No agent/environment split is needed to define it, and no goal needs to be mentioned to price it.
- Microscopic physics is reversible, and the second law of thermodynamics follows from that reversibility once dynamics are coarse-grained: a doubly stochastic Markov step cannot decrease ensemble entropy (Theorem 4.1). Global entropy reduction is impossible; all optimization is local, and the books must balance elsewhere.
- Thermodynamic laws are about information, not molecules: they bind whenever a designer cannot observe a system, cannot destroy information, and faces a conservation constraint. In Wentworth's coin world, work extraction is data compression, no work comes from a single maxentropic pool, and the fuel of every engine is negentropy, the gap between a system's entropy and its constrained maximum.
- Steering beyond blindness costs information. The Touchette–Lloyd theorem bounds any policy's entropy reduction by the blind-dynamics baseline plus the mutual information between action and environment ($\Delta H \leq \Delta H^{\max}_{\text{blind}} + I(X;A)$), so observed optimization beyond the baseline certifies the presence of modeling. The bound is a constraint, not a guarantee: information can be wasted.
- An embedded agent has exactly three ways to reduce a subsystem's entropy: dump it into the environment as heat, absorb it into memory by measurement, or spend pre-existing mutual information with the subsystem. Maxwell's demon is the second channel; its measure-then-steer operation decomposes into buying correlation and spending it.
- The Gibbs–Shannon entropy is defined relative to an exogenous, subjective ensemble, and this fails exactly on the states agents are made of: intricate knowledge gets booked at zero cost, non-concentrated ensembles yield averages true of no actual state, and atypical structure goes uncredited. Equilibrium thermodynamics is the special regime (simple, concentrated, typical ensembles) where the failure never bites.
- Algorithmic thermodynamics replaces the ensemble with Kolmogorov complexity: entropy becomes an objective property of the individual state, agreeing with the classical entropies in their domain and extending beyond it. The algorithmic second law holds per trajectory with exactly priced fluctuation allowances ($K(t-s)+\log\frac{1}{\delta}$, covering Poincaré recurrence and chance dips); deterministic simple dynamics cannot create entropy; erasure costs $k_{\mathrm{B}}T\ln 2$ joules per bit of destroyed complexity (Landauer); and the demon obeys $K(x) - K(y) \overset{+}{<} I(m(x) : x)$, the single-state form of Touchette–Lloyd. Entropy must be uncomputable for any of this to work: a computable entropy admits a perpetual-motion machine.
- For embedded agents, knowledge is endogenous: an agent's probabilistic model of its environment is physically encoded in memory, its content is the algorithmic mutual information between memory and world, and that mutual information is the consumable, perishable resource that funds optimization beyond blind baselines. Agents that know more about the world can do more to it, and thermodynamics sets the exchange rate.

A A thermodynamics of biased coins: the generalized heat engine

This appendix develops, as a self-contained worked example, the toy thermodynamic world referenced from several points in the main text (the blind baseline of Section 6, the Type-1 channel of Section 5, and the information engine of Section 8.7). It can be read at any point after Section 4. It follows Wentworth's

essay *Generalized Heat Engine*, whose goal is to take the distinctively "thermo-flavored" ideas of statistical mechanics (heat, work, engines, the impossibility of perpetual motion) and rebuild them in a setting with no physics at all, so that it becomes clear which parts of thermodynamics are really facts about information. The model also makes vivid a viewpoint, the *designer's viewpoint*, that transfers directly to thinking about agents embedded in environments they cannot fully observe.

A.1 The designer's viewpoint

Why can't a refrigerator designer simply build a machine that looks at the air molecules and removes the fast ones? Consider the designer's epistemic situation. All the microscopic dynamics of the fridge-plus-environment system are reversible, so, as we established in Section 4.1, the number of possible microstates compatible with what is known never decreases on its own. The only way to reduce uncertainty about a microstate is to observe it. But the designer is designing the machine *in advance*: deciding today how the machine will behave, with no access to the exact positions the air molecules will occupy when the machine is eventually switched on. From the designer's perspective there is uncertainty that cannot be reduced, only *moved around*. The machine itself can "observe" variables while running, but the machine is part of the total system, so its observations are just more reversible dynamics: any record the machine makes is a physical change in the machine, subject to the same conservation rules as everything else. (We will analyze exactly this maneuver, observation implemented as reversible copying, when we meet Maxwell's demon in Section 5.)

The design problem, then, is to choose transformations, in advance and sight unseen, that leave us *more* certain about some chosen variables (the inside of the refrigerator) at the cost of becoming *less* certain about others (the heat baths that power the machine). Thermodynamic-style laws apply whenever three conditions hold: the designer cannot gain information about the system (no peeking at design time), information cannot be destroyed at the lowest level (reversibility), and there is some conserved quantity constraining the transformations (energy, in physics). Wentworth's toy world has all three conditions, and nothing else.

A.2 The setup: coins, transformations, and two conservation laws

The world consists of two large pools of independent biased coins, where each coin shows 1 (heads) or 0 (tails):

- a *cold pool* $X_1^C, \ldots, X_n^C$, in which each coin is heads with probability 0.1 (entropy $h(0.1) \approx 0.47$ bits per coin, by Example 2.2); and
- a *hot pool* $X_1^H, \ldots, X_n^H$, in which each coin is heads with probability 0.2 (entropy $h(0.2) \approx 0.72$ bits per coin).

The heads-probability plays the role of temperature (the hot pool is more uncertain per coin), and the *number of heads* plays the role of energy. We may apply any transformation T that replaces the coins' values with new values computed from their old values, subject to three rules, mirroring the three conditions above:

1. **Reversibility.** T must be invertible: from the final state of all the coins (and knowledge of which transformation was applied) the initial state must be reconstructible. This is the analogue of microscopic reversibility.
2. **Conservation.** T must conserve the total number of heads, the analogue of energy conservation. Heads can be moved between coins but neither created nor destroyed on net.
3. **No peeking.** We choose T before looking at any coins, exactly as the refrigerator designer commits to a blueprint before the machine meets its environment.

To see that interesting transformations exist within these rules, here is Wentworth's example. Define T to act on three coins as follows: *if* X_1^C *shows tails, swap* X_3^H *with* X_7^H*; if* X_1^C *shows heads, change nothing.* This conserves heads (it either swaps two coins, which permutes values without changing their sum, or does nothing), and it is reversible (the control coin X_1^C is itself unchanged, so from the final state we can tell whether the swap occurred, and applying the same transformation again undoes it). Conditional

swaps like this, chained together, are the engine-builder's basic toolkit, and notice what the example already demonstrates: a transformation can *read* one part of the system and act on another, all within deterministic, reversible, conservative rules. Machines that "observe while running" are not outside the formalism; they are inside it.

In summary, we choose an invertible, heads-conserving map T from coin configurations to coin configurations, the world applies it, and we ask what such maps can accomplish.

A.3 Extracting work is a compression problem

Define a *work coin* to be a coin whose final value is heads with near-certainty (probability approaching 1 as n grows). Work coins are the analogue of stored mechanical work or a charged battery: a resource in a known, definite state, available to power other processes. *Extracting work* means choosing T so that some w designated coins end up as work coins.

The first key observation is that reversibility turns work extraction into a data compression problem. Write X for the (random) initial configuration of all the coins and $X' = T(X)$ for the final configuration. Because T is invertible, the final state determines the initial state: X' contains *all* of the information in X, so $H(X') \geq H(X)$ (in fact $H(X') = H(X)$, since invertible maps preserve entropy exactly). Now suppose w of the final coins are deterministic. Deterministic coins carry zero entropy, so all $H(X)$ bits of initial uncertainty must be carried by the remaining coins. In other words: *to extract w work coins, we must compress the information content of the entire initial state into the other coins.* Producing certainty in one place requires concentrating uncertainty elsewhere; nothing is ever erased.

A.4 No work from a single heat bath

Let us try to extract work from the hot pool alone, and watch the attempt fail; the failure is the toy-world version of the impossibility of a perpetual motion machine of the second kind (a machine that extracts work from a single-temperature heat bath).

The hot pool's n coins carry $H(X) = 0.72n$ bits of entropy. A compression scheme can in principle pack these bits into about $0.72n$ coins, leaving $0.28n$ coins deterministic. But here the conservation law steps in. Fully compressed data is incompressible, which means it looks statistically uniform: each compressed coin is heads with probability about $\frac{1}{2}$ (if the compressed coins were biased, they would be further compressible, contradicting full compression). So the $0.72n$ compressed coins would contain about $0.36n$ heads. The initial state only has $0.2n$ heads to go around, and heads are conserved. Even if every one of the would-be work coins were set to tails instead of heads, we cannot balance the books: the compression we need requires more heads than we possess. The attempt fails.

This argument generalizes well beyond coins, as follows. The hot pool's distribution (independent coins, each heads with probability 0.2) is the *maximum-entropy* distribution among all distributions with its expected number of heads; in this precise sense a heat bath is "as random as its energy allows". Suppose, in general, that a pool of variables is maxentropic subject to a constraint that fixes the value of some additive quantity $\sum_k f_k(X_k)$. If we deterministically fix the value of one variable (extract work from it), we shrink the set of values the constrained sum can take on the remaining variables, and therefore shrink the maximum entropy the remaining variables can hold. Since the initial state already saturated the maximum, the remaining variables cannot absorb all the information; compression must fail. *One cannot extract work from a system that is already at maximum entropy given its constraints.* The slack between a system's actual entropy and its constrained maximum, which following standard usage we call *negentropy*, is the only fuel there is.

A.5 Work from two heat baths at different temperatures

Now use both pools: $2n$ coins, total entropy $\approx 0.47n + 0.72n = 1.19n$ bits, total heads $\approx (0.1 + 0.2)n = 0.3n$. The crucial difference from the single-pool case is that the *joint* initial distribution is not maxentropic given the joint constraint. The maximum-entropy distribution with $0.3n$ heads spread over $2n$ coins would make every coin heads with probability 0.15; our actual state instead maintains two distinct biases, 0.1 and 0.2. That gap between actual entropy and constrained maximum entropy is negentropy, and we can spend it.

Let us compute how much work it buys. Suppose we aim for w work coins (deterministic heads). The remaining $2n - w$ coins must carry all $1.19n$ bits of initial entropy while containing the remaining $0.3n - w$ heads. To carry as much information as possible, the final distribution of those $2n - w$ coins should itself be maxentropic subject to its heads constraint, which (for large n) means independent coins, each heads with probability

$$q \;=\; \frac{0.3n - w}{2n - w},$$

giving total entropy $(2n-w)\,h(q)$ where h is the binary entropy function of Example 2.2. The largest feasible w makes this capacity exactly equal to the required $1.19n$ bits:

$$(2n - w)\; h\!\left(\frac{0.3n - w}{2n - w}\right) \;=\; 1.19n.$$

Solving numerically gives $w \approx 0.011n$. So a heads-conserving, reversible, designed-in-advance transformation *can* extract work from two heat baths at different temperatures, converting about 3.7% of the total heads (5.5% of the hot pool's heads) into work coins. This is the toy world's heat engine, and we built it with nothing but information theory.

Wentworth notes one caveat about comparing this number to physics textbooks: the efficiency computed here is not the classical Carnot efficiency, because the classical result answers a slightly different question (the optimal conversion rate *at the margin*, for engines drawing from effectively inexhaustible baths, generally consuming unequal amounts from the hot and cold sides), whereas we asked how much total work can be extracted from two fixed, finite pools. The conceptual content (work comes only from temperature *differences*, never from a single bath) is the same.

A.6 What the toy world teaches

Four lessons from this construction recur throughout the rest of the notes, so we state them explicitly.

First, thermodynamic laws are not specifically about molecules and joules. They apply whenever an agent or designer cannot gain information about a system at will, cannot destroy information at the lowest level, and operates under some conservation constraint. This is why the same laws will reappear when we analyze agents, memories, and computations.

Second, under reversibility, uncertainty is never destroyed, only moved. Every machine that makes one variable more predictable makes others less predictable.

Third, extracting work (producing variables in definite, known states) is a compression problem, and conversely, compression capacity is work capacity. This identity between "useful energy" and "compressibility" is the seed of the algorithmic viewpoint in Section 8, where it will be sharpened from a statement about distributions into a statement about individual states.

Fourth, the fuel of all such machines is negentropy: the gap between a system's entropy and the maximum entropy compatible with its constraints. A system at its constrained maximum entropy (a single heat bath) is useless as fuel, no matter how much "energy" it contains.

Everything in this section was achieved *blind*: the transformation was chosen before looking at any coins, and all the work came from statistical structure (the bias difference between pools) that was known at design time. The obvious next question is what an agent could additionally accomplish if it were allowed to *look*. That question has a precise answer, which we take up in Section 6.

Sources and further reading

These notes integrate the following sources, listed in roughly the order their material appears.

- A. Flint, *The ground of optimization*, AI Alignment Forum, 2020. The source for optimizing systems (Definition 3.1), the basin/target/robustness vocabulary, the duality and retargetability axes, and the examples of Section 3.
- E. Yudkowsky, *Measuring optimization power*, LessWrong, 2008. The source for measuring optimization as the improbability of the achieved outcome (Section 3.4).

- J. Wentworth, *Generalized heat engine*, LessWrong, 2020. The source for all of Appendix A: the designer's viewpoint, the biased-coin world, work extraction as compression, and the two-bath engine. His decomposition of expected utility maximization (Remark 3.5) appears in *Utility maximization = description length minimization*, LessWrong, 2021.

- A. Harwood and A. Altair, *When bits of optimization imply bits of modeling: the Touchette–Lloyd theorem*, LessWrong, 2025. The pedagogical source for Section 6, including the guessing game, the blind/sighted vocabulary, the headphone analogy, and the insufficiency caveats. The underlying theorem is from H. Touchette and S. Lloyd, *Information-theoretic approach to the study of control systems*, Physica A 331:140–172, 2004 (Theorem 10), with antecedents in their 2000 paper *Information-theoretic limits of control* and Lloyd's 1989 work on Maxwell's demon.

- Daniel C and A. Ebtekar, *Algorithmic thermodynamics and three types of optimization*, AI Alignment Forum, 2025. The central organizing source for these notes: the three types of optimization (Section 5), the argument that entropy should objectively measure optimization capacity (Section 7), the algorithmic refinements of the three types, and the embedded-agency synthesis (Section 9).

- A. Ebtekar and M. Hutter, *Foundations of algorithmic thermodynamics*, Physical Review E, 2025 (arXiv:2308.06927). The formal backbone of Sections 4 and 8: Markovian coarse-grainings and the multibaker construction, Gács' coarse-grained algorithmic entropy, Levin's randomness conservation and the algorithmic second law with its $K(t-s)+\log\frac{1}{\delta}$ allowance, the algorithmic Jarzynski and Landauer inequalities, the Landauer-cost/EP-cost decomposition and the misconception corrections, the costs of randomization, computation, and measurement, the exact Maxwell's demon analysis including the partial-measurement bound (2), the ensemble-versus-state comparison of Section 7 (including the robot-battery and bookshelf examples and Zurek's identity), and the information engine of Section 8.7.

For the broader agent foundations context assumed in Section 1 (true names and Goodhart's law, reflective stability, embedded agency, selection theorems), see the Agent Foundations slide deck accompanying this module.